UNIT I

Introduction to OR & Linear Programming

Handwritten study notes | B.Tech | Operations Research

Syllabus topics covered in this unit:

- Definition of OR, classification of models, modelling, methods of solving
- Limitations and applications of OR models
- LP: formulation, graphical method, simplex, Big-M, two-phase method
- Special cases: degeneracy, infeasibility, multiple optima, unboundedness
- Concept of duality and the dual theorems

1. Introduction to Operations Research

Origin: OR began in the Second World War (1939-40). British scientists (Blackett's 'Circus') were asked to find the best way of using limited radar, convoys and anti-aircraft resources. After the war the methods moved into industry, business and government. In India, OR started in 1949 at the Regional Research Laboratory, Hyderabad; the Operational Research Society of India was formed in 1957.

Definitions (learn any two)

Churchman, Ackoff & Arnoff (1957): OR is the application of scientific methods, techniques and tools to problems involving the operations of a system, so as to provide those in control of the system with optimum solutions to the problems.

Morse & Kimball: OR is a scientific method of providing executive departments with a quantitative basis for decisions regarding the operations under their control.

Characteristics / features of OR

- **Interdisciplinary team approach** - mathematicians, engineers, economists, statisticians work together
- **Scientific method** - observation, hypothesis, experiment, conclusion
- **Systems (holistic) approach** - the whole organisation is studied, not one department
- **Use of models** - real problem is represented by a mathematical model
- **Optimisation** - aim is the best (optimum) decision under given limits
- **Quantitative and computer based** - uses data, algorithms and software

Phases (steps) of an OR study

1. Define problem → 2. Build model → 3. Solve model → 4. Test validate → 5. Implement & control

- *Define the problem: objective, alternatives, limitations of the system*
- *Construct the model (mathematical equations or inequalities) from the data*
- *Solve the model with a suitable technique (simplex, MODI, etc.) to get a solution*
- *Validate - check whether results predict the real system correctly; update model*
- *Implement the solution and control it, since conditions keep changing*

2. Classification of OR Models

*A **model** is an idealised, simplified representation of a real-life situation. It shows the relation between the controllable (decision) variables, uncontrollable variables and the objective.*

Basis of classification	Types	Example
By structure	Iconic / Analogue / Symbolic	Toy plane; graph; equation
By nature of environment	Deterministic / Probabilistic	LP model; queueing model
By time	Static / Dynamic	EOQ; dynamic programming
By purpose	Descriptive / Predictive / Prescriptive	Survey; forecast; LP
By method of solution	Analytical / Simulation	Calculus; Monte-Carlo
By generality	General / Specific	Simplex; sales-forecast

- *Iconic model: physical look-alike, scaled up or down (globe, scale model of a dam, aircraft)*
- *Analogue model: one property stands for another (contour map, thermometer, graphs, speedometer)*
- *Symbolic (mathematical) model: letters and symbols for variables, equations for relations.* ***Most common in OR.***
- *Deterministic: all data known with certainty. Probabilistic (stochastic): data follow probability laws*
- *Static: decision for one period only. Dynamic: decisions are linked over several periods*

Modelling - qualities of a good model

- *Simple enough to be solved, yet realistic enough to be useful*
- *Easy to modify and update; data should be easily available*
- *Includes only the **important** variables, ignoring minor details*
- *Steps: identify variables -> state objective -> write constraints -> solve -> test & revise*

Methods of solving OR models

- *Analytical method: classical mathematics (calculus, algebra, matrices) gives a direct formula, e.g. EOQ*
- *Iterative (numerical) method: start with a trial solution, improve step by step till the optimum, e.g. simplex, MODI*
- *Monte-Carlo (simulation) method: random sampling is used to imitate the system when equations are too complex*
- *Heuristic method: rules of thumb that give a good (not always optimal) answer quickly*

3. Limitations and Applications of OR

Limitations

- Models are only approximations; real life has factors that cannot be quantified (human feelings)
- Collecting accurate data is costly and time-consuming; wrong data gives wrong decisions
- Large problems need expensive computers and trained experts
- OR gives a **recommendation**, it cannot replace managerial judgement
- Managers may resist change; implementation may fail even if the model is correct

Applications

Area	Typical use of OR
Production / Inventory	Product mix, scheduling, EOQ, plant location
Marketing	Media selection, pricing, sales-force allocation
Finance	Capital budgeting, portfolio selection, cash flow
Transportation	Least cost distribution, routing, assignment of crews
Personnel	Manpower planning, recruitment, shift scheduling
Defence / Govt / Health	Resource allocation, hospital queues, project planning

Main OR techniques: LP, transportation, assignment, game theory, queuing, inventory, replacement, sequencing, dynamic programming, simulation, PERT/CPM.

4. Linear Programming (LP) - Formulation

LP is a mathematical technique for allocating **limited resources** among competing activities so that a linear objective function is optimised (maximised profit or minimised cost) subject to linear constraints.

Assumptions of LP

- *Linearity / proportionality* - contribution and resource use are proportional to the quantity
- *Additivity* - total = sum of individual parts *Divisibility* - fractional values allowed
- *Certainty* - all coefficients are known constants *Non-negativity* - variables are $x_j \geq 0$

General form of an LP model

Maximise (or Minimise) $Z = c_1x_1 + c_2x_2 + \ldots + c_nx_n$ *(objective function)*

subject to $a_{i1}x_1 + a_{i2}x_2 + \ldots + a_{in}x_n \ (\leq , = , \geq) \ b_i, \quad i = 1, 2, \ldots, m$ *(constraints)*

$x_1, x_2, \ldots, x_n \geq 0$ *(non-negativity)*

Terms to remember

- **Feasible solution:** satisfies all constraints. **Optimal solution:** feasible and gives best Z
- **Basic solution:** n - m variables set to zero, rest solved. **BFS:** basic solution that is also feasible
- **Feasible region:** common area of all constraints; it is a **convex set**

Steps in formulation

(1) Identify decision variables (2) write objective function (3) list resource constraints from the data (4) add non-negativity.

Example 1 - Product mix (maximisation)

A firm makes products P_1 and P_2. P_1 needs 2 hrs on machine A and 4 hrs on machine B per unit; P_2 needs 3 hrs on A and 3 hrs on B. Machine A has 60 hrs and B has 96 hrs available. Profit is Rs 40 per unit of P_1 and Rs 35 per unit of P_2. Formulate the LP.

Let x_1, x_2 = number of units of P_1, P_2 produced.
Maximise $Z = 40x_1 + 35x_2$
subject to $2x_1 + 3x_2 \le 60$ (machine A)
$4x_1 + 3x_2 \le 96$ (machine B)
$x_1, x_2 \ge 0$

Example 2 - Diet problem (minimisation)

Food A costs Rs 6/unit and gives 2 units protein, 1 unit vitamin. Food B costs Rs 4/unit and gives 1 unit protein, 3 units vitamin. Daily need: at least 12 protein and 15 vitamin. Minimise cost.

x_1, x_2 = units of food A, B. Minimise $Z = 6x_1 + 4x_2$
s.t. $2x_1 + x_2 \ge 12$ (protein), $x_1 + 3x_2 \ge 15$ (vitamin), $x_1, x_2 \ge 0$

Example 3 - Blending / investment type

Rs 1,00,000 is to be invested in bonds (x_1, return 8%) and shares (x_2, return 12%). Not more than 60% in shares and at least 20% in bonds.

Max $Z = 0.08x_1 + 0.12x_2$ s.t. $x_1 + x_2 \le 1{,}00{,}000$; $x_2 \le 60{,}000$; $x_1 \ge 20{,}000$; $x_1, x_2 \ge 0$

Standard form - slack, surplus and artificial variables

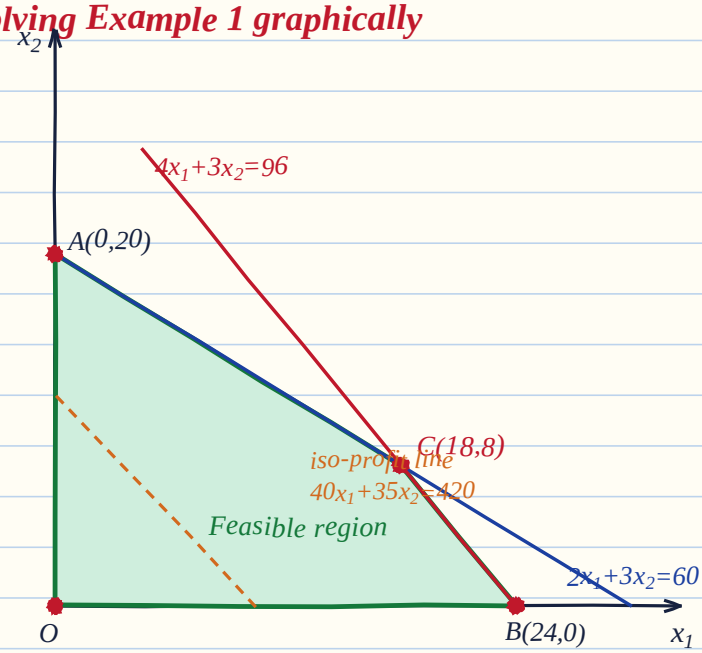
Constraint type	Add / subtract	Cost in Z (Max)	Cost in Z (Min)	Starting basic?
≤	+ slack s	0	0	Yes (s)
≥	- surplus s, + artificial a	s: 0, a: -M	s: 0, a: +M	Yes (a)
=	+ artificial a	-M	+M	Yes (a)

Rule: RHS of every constraint must be non-negative. If $b < 0$ multiply the whole constraint by -1 (inequality reverses). Slack = unused resource; surplus = excess over the minimum.

5. Graphical Method (two variables)

- Step 1: convert each inequality to an equation and plot the line
- Step 2: shade the side satisfying the inequality; the common area is the **feasible region**
- Step 3: find the coordinates of all corner (extreme) points
- Step 4: compute Z at each corner point - best value is optimal (**corner point theorem**), or move an iso-profit/iso-cost line parallel to itself

Solving Example 1 graphically



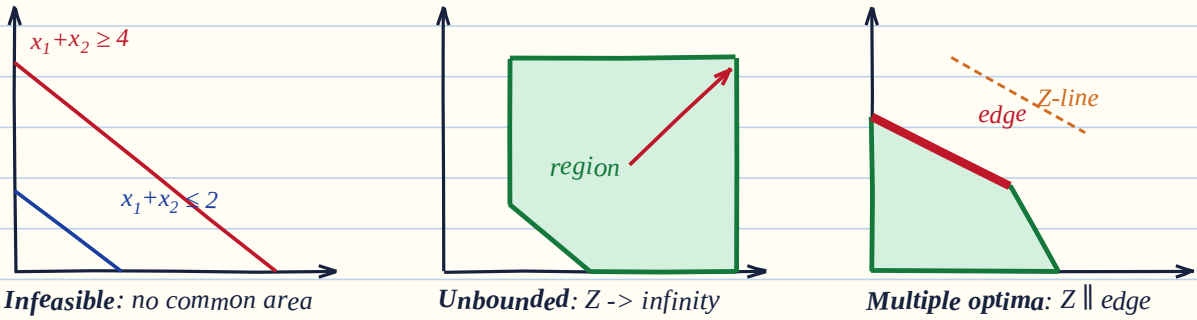
Corner pt	$Z = 40x_1+35x_2$
O (0,0)	0
B (24,0)	960
C (18,8)	**1000 <- max**
A (0,20)	700

Optimum: $x_1=18$, $x_2=8$

Max Z = Rs 1000

C = intersection of both lines:
subtract: $2x_1=36 \Rightarrow x_1=18$, $x_2=8$

Special cases seen on the graph



Infeasible: no common area — **Unbounded**: Z -> infinity — **Multiple optima**: Z ∥ edge

Note: multiple optimum = whole edge optimal, so infinitely many optimal solutions. Check by hand: Max $Z = 2x_1+4x_2$; $x_1+2x_2 \leq 5$; $x_1+x_2 \leq 4$ gives $Z = 10$ at both (0, 2.5) and (3, 1).

6. Simplex Method

Used when there are more than two variables. It moves from one basic feasible solution (corner) to an adjacent better one till no improvement is possible.

Algorithm (maximisation)

- Step 1: write standard form (add slacks); make all $b_i \geq 0$. Initial BFS: slacks in the basis
- Step 2: build the table; compute $Z_j = \sum C_B \cdot a_j$ and $C_j - Z_j$ (net evaluation row)
- Step 3: **Optimality test** - if all $(C_j - Z_j) \leq 0$ (Max) the solution is optimal. For Min: all ≥ 0
- Step 4: **Entering variable** = largest positive $(C_j - Z_j)$ -> key column

- Step 5: **Leaving variable** = minimum ratio b_i / (key-column entry > 0) -> key row; pivot = intersection
- Step 6: new pivot row = old row / pivot; other rows: $R_{new} = R_{old}$ - (key column entry)×(new pivot row). Repeat from Step 2

Solving Example 1 by simplex

Standard form: Max $Z = 40x_1 + 35x_2 + 0s_1 + 0s_2$; $2x_1+3x_2+s_1 = 60$; $4x_1+3x_2+s_2 = 96$.

Iteration 0

	Cj	40	35	0	0		
Cb	**Basis**	**x1**	**x2**	**s1**	**s2**	**b**	**Ratio**
0	s1	2	3	1	0	60	30
0	s2	4	3	0	1	96	24
	Zj	0	0	0	0	0	
	Cj-Zj	40	35	0	0		

Entering = **x1**, Leaving = **s2**, Pivot element = **4**

Iteration 1

	Cj	40	35	0	0		
Cb	**Basis**	**x1**	**x2**	**s1**	**s2**	**b**	**Ratio**
0	s1	0	3/2	1	-1/2	12	8
40	x1	1	3/4	0	1/4	24	32
	Zj	40	30	0	10	960	
	Cj-Zj	0	5	0	-10		

Entering = **x2**, Leaving = **s1**, Pivot element = **3/2**

Final table (iteration 2)

	Cj	40	35	0	0		
Cb	**Basis**	**x1**	**x2**	**s1**	**s2**	**b**	**Ratio**
35	x2	0	1	2/3	-1/3	8	
40	x1	1	0	-1/2	1/2	18	
	Zj	40	35	10/3	25/3	1000	
	Cj-Zj	0	0	-10/3	-25/3		

All Cj - Zj satisfy the optimality condition, so STOP. Optimal reached.

Result: $x_1 = 18, x_2 = 8, s_1 = s_2 = 0$, Max $Z = 1000$ (same as graphical).
Shadow prices (Z_j under slacks): machine A = 10/3, machine B = 25/3 per extra hour.

7. Big-M Method (Charne's penalty method)

Used when constraints are of type ≥ or = , because there is no obvious starting basis. Artificial variables are added and given a huge penalty M (M is larger than any number in the problem).

- Min problem: artificial cost = +M. Max problem: artificial cost = -M
- Solve like simplex. Once an artificial leaves the basis, its column may be dropped
- Optimal with an artificial still in basis at positive level => problem is INFEASIBLE

Example 4: Min $Z = 4x_1 + x_2$; $3x_1+x_2 = 3$; $4x_1+3x_2 \geq 6$; $x_1+2x_2 \leq 4$

Standard form: $3x_1+x_2+a_1 = 3$; $4x_1+3x_2-s_2+a_2 = 6$; $x_1+2x_2+s_3 = 4$. Objective: Min $Z = 4x_1+x_2+M\,a_1+M\,a_2$.

Table 1

	Cj	4	1	M	0	M	0		
Cb	**Basis**	**x1**	**x2**	**a1**	**s2**	**a2**	**s3**	**b**	**Ratio**
M	a1	3	1	1	0	0	0	3	1
M	a2	4	3	0	-1	1	0	6	3/2
0	s3	1	2	0	0	0	1	4	4
	Zj	7M	4M	M	-M	M	0	9M	
	Cj-Zj	4 - 7M	1 - 4M	0	M	0	0		

Entering = **x1**, Leaving = **a1**, Pivot element = **3**

Table 2

	Cj	4	1	M	0	M	0		
Cb	**Basis**	**x1**	**x2**	**a1**	**s2**	**a2**	**s3**	**b**	**Ratio**
4	x1	1	1/3	1/3	0	0	0	1	3
M	a2	0	5/3	-4/3	-1	1	0	2	6/5
0	s3	0	5/3	-1/3	0	0	1	3	9/5
	Zj	4	4/3 + 5/3M	4/3 - 4/3M	-M	M	0	4 + 2M	
	Cj-Zj	0	-1/3 - 5/3M	-4/3 + 7/3M	M	0	0		

Entering = **x2**, Leaving = **a2**, Pivot element = **5/3**

Table 3

	Cj	4	1	M	0	M	0		
Cb	**Basis**	**x1**	**x2**	**a1**	**s2**	**a2**	**s3**	**b**	**Ratio**
4	x1	1	0	3/5	1/5	-1/5	0	3/5	3
1	x2	0	1	-4/5	-3/5	3/5	0	6/5	
0	s3	0	0	1	1	-1	1	1	1
	Zj	4	1	8/5	1/5	-1/5	0	18/5	
	Cj-Zj	0	0	-8/5 + M	-1/5	1/5 + M	0		

*Entering = **s2**, Leaving = **s3**, Pivot element = **1***

Table 4

	Cj	4	1	M	0	M	0		
Cb	**Basis**	**x1**	**x2**	**a1**	**s2**	**a2**	**s3**	**b**	**Ratio**
4	x1	1	0	2/5	0	0	-1/5	2/5	
1	x2	0	1	-1/5	0	0	3/5	9/5	
0	s2	0	0	1	1	-1	1	1	
	Zj	4	1	7/5	0	0	-1/5	17/5	
	Cj-Zj	0	0	-7/5 + M	0	M	1/5		

All Cj - Zj satisfy the optimality condition, so STOP. Optimal reached.

Optimal: $x_1 = 2/5, x_2 = 9/5, s_2 = 1$, Min Z = 17/5 = 3.4

Check: 3(0.4) + 1.8 = 3 (ok); 4(0.4)+3(1.8) = 7 ≥ 6 (ok); 0.4 + 3.6 = 4 (ok).

8. Two-Phase Simplex Method

Removes the use of M (avoids big numbers and computer round-off). The same problem is solved in two phases.

Phase I: replace objective by Min W = sum of artificial variables (cost 1 for each a, 0 for all others). Solve by simplex.

- if Min W = 0 -> a feasible solution exists, go to Phase II.
- if Min W > 0 -> problem is infeasible, stop.

Phase II: delete artificial columns, restore the ORIGINAL objective coefficients, and continue simplex from the final Phase I table.

Example 4 again by two-phase method

Phase I: Min W = $a_1 + a_2$

Phase I - table 1

	Cj	0	0	1	0	1	0		
Cb	**Basis**	**x1**	**x2**	**a1**	**s2**	**a2**	**s3**	**b**	**Ratio**
1	a1	3	1	1	0	0	0	3	1
1	a2	4	3	0	-1	1	0	6	3/2
0	s3	1	2	0	0	0	1	4	4
	Zj	7	4	1	-1	1	0	9	
	Cj-Zj	-7	-4	0	1	0	0		

*Entering = **x1**, Leaving = **a1**, Pivot element = **3***

Phase I - table 2

	Cj	0	0	1	0	1	0		
Cb	**Basis**	**x1**	**x2**	**a1**	**s2**	**a2**	**s3**	**b**	**Ratio**
0	x1	1	1/3	1/3	0	0	0	1	3
1	a2	0	5/3	-4/3	-1	1	0	2	6/5
0	s3	0	5/3	-1/3	0	0	1	3	9/5
	Zj	0	5/3	-4/3	-1	1	0	2	
	Cj-Zj	0	-5/3	7/3	1	0	0		

Entering = **x2**, Leaving = **a2**, Pivot element = **5/3**

Phase I - table 3

	Cj	0	0	1	0	1	0		
Cb	**Basis**	**x1**	**x2**	**a1**	**s2**	**a2**	**s3**	**b**	**Ratio**
0	x1	1	0	3/5	1/5	-1/5	0	3/5	
0	x2	0	1	-4/5	-3/5	3/5	0	6/5	
0	s3	0	0	1	1	-1	1	1	
	Zj	0	0	0	0	0	0	0	
	Cj-Zj	0	0	1	0	1	0		

All Cj - Zj satisfy the optimality condition, so STOP. Optimal reached.

Min W = 0 and no artificial is in the basis => feasible solution found. Go to Phase II.

Phase II: Min $Z = 4x_1 + x_2$ (artificial columns deleted)

Phase II - table 1

	Cj	4	1	0	0		
Cb	**Basis**	**x1**	**x2**	**s2**	**s3**	**b**	**Ratio**
4	x1	1	0	1/5	0	3/5	3
1	x2	0	1	-3/5	0	6/5	
0	s3	0	0	1	1	1	1
	Zj	4	1	1/5	0	18/5	
	Cj-Zj	0	0	-1/5	0		

Entering = **s2**, Leaving = **s3**, Pivot element = **1**

Phase II - table 2

	Cj	4	1	0	0		
Cb	**Basis**	**x1**	**x2**	**s2**	**s3**	**b**	**Ratio**
4	x1	1	0	0	-1/5	2/5	
1	x2	0	1	0	3/5	9/5	
0	s2	0	0	1	1	1	
	Zj	4	1	0	-1/5	17/5	
	Cj-Zj	0	0	0	1/5		

All Cj - Zj satisfy the optimality condition, so STOP. Optimal reached.

Same answer: $x_1 = 2/5, x_2 = 9/5, Z = 17/5$.

Point	Big-M	Two-Phase
Penalty	M inside the table (symbolic)	No M; two separate LPs
Work	Single table sequence	Phase I + Phase II
Risk	Round-off with large M	More tables, but safer
Infeasible test	Artificial positive in basis	Min W > 0 in Phase I

9. Special Cases in LP (simplex view)

Case	How to recognise in the table	Meaning
Degeneracy	Tie in minimum ratio; a basic variable = 0	Possible cycling
Infeasibility	Optimal table has artificial in basis with value > 0	No feasible region
Multiple optima	Non-basic variable has Cj-Zj = 0 at optimum	Infinitely many optima
Unboundedness	Entering column has no positive entry (no ratio)	Z -> infinity

(a) Degeneracy

A BFS is degenerate if one or more basic variables are zero. It arises when two rows tie for the minimum ratio; the next table then has a zero in b-column. Danger: cycling (same bases repeat). Cure: break the tie by **Bland's rule** (choose the lowest index) or by perturbing b-values slightly.

Example: Max $Z = 3x_1 + 9x_2$; $x_1+4x_2 \le 8$; $x_1+2x_2 \le 4$.
First ratios: 8/4 = 2 and 4/2 = 2 (tie!). x_2 enters; whichever row leaves, the other slack stays basic at ZERO => degenerate.
Optimum Z = 18 at $x_1 = 0, x_2 = 2$. In graph: three lines pass through one point (redundant constraint).

(b) Infeasibility

Example: Max $Z = 3x_1+2x_2$; $x_1+x_2 \le 2$; $x_1+x_2 \ge 4$. No point satisfies both. In Big-M, the final table keeps an artificial variable at a positive value ($a_2 = 2 > 0$).

(c) Multiple optimal solutions

When a non-basic variable has $C_j - Z_j = 0$ in the optimal table, entering it gives another corner with the same Z. If X^1, X^2 are optimal then $X = \lambda X^1 + (1-\lambda)X^2$, $0 \le \lambda \le 1$ is also optimal. (see example in section 5.)

(d) Unbounded solution

Example: Max $Z = 3x_1+2x_2$; $x_1-x_2 \le 1$; $3x_1+x_2 \ge 3$. The feasible region extends to infinity along x_2 and Z grows without limit. Wrong formulation in practice - re-check constraints.

10. Concept of Duality

Every LP (called **primal**) has a companion LP called its **dual**. Both use the same data (A, b, c). Solving one gives the solution of the other; the dual of the dual is the primal.

Primal (Max): $\text{Max } Z = c\,x \quad \text{s.t. } A\,x \le b,\ x \ge 0$

Dual (Min): $\text{Min } W = b^T y \quad \text{s.t. } A^T y \ge c^T,\ y \ge 0$

Rules for converting primal to dual

Primal (Max)	Dual (Min)
Objective coefficients c_j	RHS constants of dual constraints
RHS constants b_i	Objective coefficients of dual
i-th constraint $\le$	i-th dual variable $y_i \ge 0$
i-th constraint $\ge$	$y_i \le 0$ (or multiply by -1 first)
i-th constraint $=$	y_i unrestricted in sign
Variable $x_j \ge 0$	j-th dual constraint is $\ge$
Variable x_j unrestricted	j-th dual constraint is $=$
m constraints, n variables	n constraints, m variables

Example: dual of Example 2 (diet problem)

Primal: Min $Z = 6x_1+4x_2$; $2x_1+x_2 \ge 12$; $x_1+3x_2 \ge 15$; $x \ge 0$

Dual: Max $W = 12y_1 + 15y_2$; $2y_1 + y_2 \le 6$; $y_1 + 3y_2 \le 4$; $y_1, y_2 \ge 0$

Solving: primal $x_1 = 4.2$, $x_2 = 3.6$, $Z = 39.6$; dual $y_1 = 2.8$, $y_2 = 0.4$, $W = 39.6$ -> **$Z = W$**

Check with Example 1: its dual is Min $W = 60y_1+96y_2$; $2y_1+4y_2 \ge 40$; $3y_1+3y_2 \ge 35$. With $y = (10/3, 25/3)$: $W = 200+800 = 1000 = Z$. These y-values are the shadow prices read from the Z_j row.

Duality theorems

- *Weak duality:* for any feasible x (primal Max) and y (dual Min), $c\,x \le b^T y$
- *Strong (fundamental) duality:* if one problem has an optimal solution so does the other and Max Z = Min W
- *Existence theorem:* if primal is unbounded the dual is infeasible; if primal is infeasible the dual is unbounded or infeasible
- *Complementary slackness:* x_j·(slack of dual j-th constraint) = 0 and y_i·(slack of primal i-th constraint) = 0 at optimum

Why study the dual? / Economic meaning

- y_i = *shadow price*: change in optimal Z per unit increase in resource b_i
- If a resource has slack > 0, its shadow price is zero (not scarce)
- Dual may be easier: more constraints than variables -> solve dual instead (fewer rows)
- Basis of dual simplex method and sensitivity (post-optimality) analysis

11. Quick Revision & Important Questions

1. Define OR; list phases; classify models. 2. Limitations & applications of OR.
3. Formulate and solve by graphical method (corner points). 4. Simplex algorithm with example.
5. Big-M vs Two-phase method with problem. 6. Explain degeneracy, infeasibility, unbounded, multiple optima.
7. Write dual of a given LP; state duality theorems.

Formula sheet: $Z_j = \sum C_B\, a_{ij}$; Net evaluation = $C_j - Z_j$; Ratio = b_i/a_{ik} ($a_{ik} > 0$); Max optimal when all $C_j - Z_j \le 0$, Min optimal when all ≥ 0; Max Z = Min W.

UNIT II

Transportation & Assignment Problems

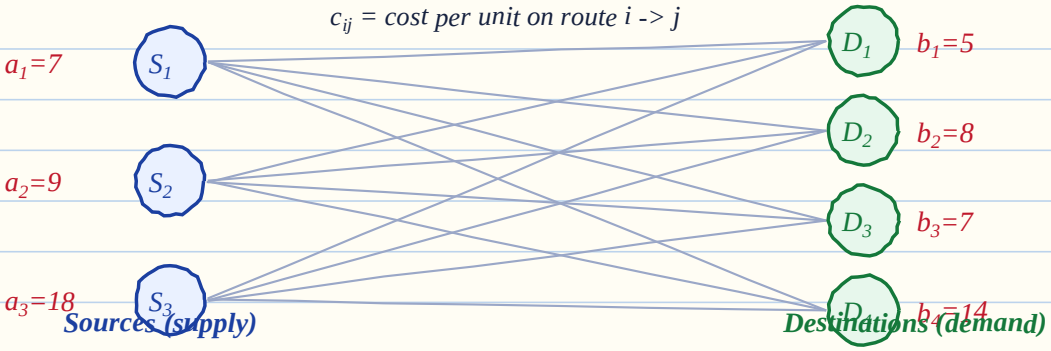
Handwritten study notes | B.Tech | Operations Research

Syllabus topics covered in this unit:

- Transportation problem: formulation, IBFS by NWC, Least Cost and VAM
- Optimality test by the MODI method
- Special cases: unbalanced and degenerate transportation problems
- Assignment problem: formulation, Hungarian method
- Traveling salesman problem

1. Transportation Problem (TP) - Formulation

The TP is a special LP in which a **single commodity** is moved from m sources (factories/warehouses) to n destinations (markets) at **minimum total transportation cost**, given the supply at each source and the demand at each destination.



Mathematical model

Minimise $Z = \sum_{i=1}^{m} \sum_{j=1}^{n} c_{ij} x_{ij}$ (x_{ij} = units sent from source i to destination j)

Supply: $\sum_j x_{ij} = a_i$ $(i = 1..m)$ Demand: $\sum_i x_{ij} = b_j$ $(j = 1..n)$ $x_{ij} \geq 0$

Balanced TP: $\sum a_i = \sum b_j$. Number of variables = m×n, constraints = m + n.

Important facts

- A **basic feasible solution** has exactly $m + n - 1$ allocated (basic) cells; the allocations must not form a closed loop
- Always feasible & bounded when balanced. All coefficients in constraints are 0 or 1, so solutions are integers if a_i, b_j are integers
- Steps: (1) check balance (2) find **initial BFS** (NWC, LCM, VAM) (3) test optimality (**MODI**) (4) improve till optimal

Formulation example (word problem)

Two plants P1, P2 (capacity 40, 60 units) supply three markets M1, M2, M3 (demand 30, 35, 35). Unit costs (Rs): P1 -> 6, 8, 4 and P2 -> 5, 7, 9.

Let x_{ij} = units sent from plant i to market j. Min $Z = 6x_{11}+8x_{12}+4x_{13}+5x_{21}+7x_{22}+9x_{23}$

Supply: $x_{11}+x_{12}+x_{13} = 40$; $x_{21}+x_{22}+x_{23} = 60$.

Demand: $x_{11}+x_{21} = 30$; $x_{12}+x_{22} = 35$; $x_{13}+x_{23} = 35$; all $x_{ij} \geq 0$.

Total supply = 100 = total demand, hence balanced. Basic cells needed = 2 + 3 - 1 = 4.

Data used for all worked examples (cost per unit in Rs)

	D1	D2	D3	D4	Supply
S1	19	30	50	10	7
S2	70	30	40	60	9
S3	40	8	70	20	18
Demand	5	8	7	14	34 = 34

Total supply 34 = total demand 34, so the problem is **balanced**. Basic cells needed = 3 + 4 - 1 = 6.

Solution of the formulation example: VAM allocation, then MODI test.

	M1	M2	M3	Supply	u_i
P1	6, Δ=0	8, 5	4, 35	40	0
P2	5, 30	7, 30	9, Δ=6	60	-1
Demand	30	35	35		
v_j	6	8	4		

VAM cost = 540

u_1=0 gives v and u_2

Δ values shown in cells

All Δ ≥ 0 -> optimal

Optimal plan: P1->M2: 5, P1->M3: 35, P2->M1: 30, P2->M2: 30. Minimum cost = **Rs 540**.

2. Initial Basic Feasible Solution

(a) North-West Corner Rule (NWCR)

- Start at the **top-left (north-west)** cell. Allocate min(supply, demand)
- Cross out the exhausted row or column; if a row is finished move **down**, if a column is finished move **right**
- Repeat till all supplies and demands are met. Cost values are ignored, so it is fast but usually **poor**

	D_1	D_2	D_3	D_4	Supply
S_1	19 **5**	30 **2**	50	10	7
S_2	70	30 **6**	40 **3**	60	9
S_3	40	8	70 **4**	20 **14**	18
Demand	5	8	7	14	

Allocations:
(1,1)=5, (1,2)=2
(2,2)=6, (2,3)=3
(3,3)=4, (3,4)=14
Cost = 1015

Z = 5(19) + 2(30) + 6(30) + 3(40) + 4(70) + 14(20) = 95+60+180+120+280+280 = ***Rs 1015****.*
Allocated cells = 6 = m+n-1, hence non-degenerate.

(b) Least Cost Method (LCM / matrix minima)

- *Select the cell with the* ***smallest unit cost*** *in the whole table; allocate as much as possible*
- *Cross out the satisfied row/column; repeat on the remaining matrix. Ties: choose the cell that allows the larger allocation*

	D_1	D_2	D_3	D_4	Supply
S_1	19	30	50	10 **7**	7
S_2	70 **2**	30	40 **7**	60	9
S_3	40 **3**	8 **8**	70	20 **7**	18
Demand	5	8	7	14	

Order of allocation:
(3,2)=8 (cost 8)
(1,4)=7 (cost 10)
(3,4)=7, (2,3)=7
(3,1)=3, (2,1)=2
Cost = 814

Z = 7(10) + 2(70) + 7(40) + 3(40) + 8(8) + 7(20) = 70+140+280+120+64+140 = ***Rs 814****.*
Better than NWCR (1015) since cost is considered.

(c) Vogel's Approximation Method (VAM / penalty method)

Gives the best starting solution; very often optimal or very close to it.

- *For every row and column find the* ***penalty = difference between the two smallest costs***
- *Select the row/column with the* ***largest penalty****; allocate max possible in its* ***least cost cell***
- *Cross out the satisfied row/column, recompute penalties for the remaining table, repeat. Ties in penalty: pick least cost cell*

Step	Row penalties S1 S2 S3	Column penalties D1 D2 D3 D4	Max	Allocate
1	9 10 12	21 22 10 10	22	x(S3,D2) = 8
2	9 20 20	21 - 10 10	21	x(S1,D1) = 5
3	40 20 50	- - 10 10	50	x(S3,D4) = 10
4	40 20 -	- - 10 50	50	x(S1,D4) = 2
5	- 20 -	- - 40 60	60	x(S2,D4) = 2
6	- 40 -	- - 40 -	40	x(S2,D3) = 7

	D_1	D_2	D_3	D_4	Supply
S_1	19 **5**	30	50	10 **2**	7
S_2	70	30	40 **7**	60 **2**	9
S_3	40	8 **8**	70	20 **10**	18
Demand	5	8	7	14	

VAM allocations:
(1,1)=5, (1,4)=2
(2,3)=7, (2,4)=2
(3,2)=8, (3,4)=10
Cost = 779

$Z = 5(19) + 2(10) + 7(40) + 2(60) + 8(8) + 10(20) = 95+20+280+120+64+200 =$ **Rs 779**.

Method	Cost (Rs)	Remark
North-West Corner	1015	Ignores costs; easiest
Least Cost	814	Uses costs; better
VAM	779	Best IBFS (optimum is 743)

3. Optimality Test - MODI (u-v) Method

MODI = Modified Distribution Method. It tests whether the current BFS is optimal using ***dual variables*** u_i *(rows) and* v_j *(columns).*

Step 1: For every ***allocated*** *cell:* $u_i + v_j = c_{ij}$. *Put* $u_1 = 0$ *(any one value) and solve for all u, v.*
Step 2: For every ***unallocated*** *cell find the opportunity cost* $\Delta_{ij} = c_{ij} - (u_i + v_j)$.
Step 3: If all $\Delta_{ij} \geq 0$ -> ***OPTIMAL*** *(*$\Delta = 0$ *for an empty cell means alternate optima).*
Step 4: Otherwise choose the most negative Δ *(entering cell), draw a closed loop through allocated cells with alternate + and -.*
Step 5: θ *= smallest allocation on the minus cells. Add* θ *at + cells, subtract at - cells. Go to Step 1.*

Applying MODI to the VAM solution (cost 779)

	D_1	D_2	D_3	D_4	Supply	u_i
S_1	19 **5**	30 Δ=32	50 Δ=60	10 **2**	7	0
S_2	70 Δ=1	30 Δ=-18	40 **7**	60 **2**	9	50
S_3	40 Δ=11	8 **8**	70 Δ=70	20 **10**	18	10
Demand	5	8	7	14		
v_j	19	-2	-10	10		

u_1=0 => v_1=19, v_4=10
u_2=50 => v_3=-10
u_3=10 => v_2=-2
$\Delta = c - (u+v)$
Δ_{22} = -18 (<0)

$\Delta_{22} = 30 - (50 - 2) = -18$ is the only negative value, so the solution is **not optimal**. Cell (S2,D2) enters.

	D_1	D_2	D_3	D_4	Supply
S_1	19 **5**	30	50	10 **2**	7
S_2	70	30 +	40 **7**	60 **2** -	9
S_3	40	8 **8** -	70	20 **10** +	18
Demand	5	8	7	14	

Closed loop:
(2,2)+ -> (2,4)-
-> (3,4)+ -> (3,2)- -> back
$\theta = min(2, 8) = 2$

Minus cells: (2,4) = 2 and (3,2) = 8, so $\theta = 2$. New allocations: $x_{22} = 2$, $x_{24} = 0$ (leaves), $x_{34} = 12$, $x_{32} = 6$.

	D_1	D_2	D_3	D_4	Supply	u_i
S_1	19 **5**	30 Δ=32	50 Δ=42	10 **2**	7	0
S_2	70 Δ=19	30 **2**	40 **7**	60 Δ=18	9	32
S_3	40 Δ=11	8 **6**	70 Δ=52	20 **12**	18	10
Demand	5	8	7	14		
v_j	19	-2	8	10		

u=(0, 32, 10)
v=(19, -2, 8, 10)
All Δ ≥ 0 -> OPTIMAL
Z = 743

Optimal solution: $x_{11}=5$, $x_{14}=2$, $x_{22}=2$, $x_{23}=7$, $x_{32}=6$, $x_{34}=12$.
$Z = 5(19)+2(10)+2(30)+7(40)+6(8)+12(20) = 95+20+60+280+48+240 =$ **Rs 743** (improved from 779).

Check: number of allocated cells must remain $m+n-1 = 6$ after every iteration. One allocated cell that became zero (x_{24}) leaves the basis.

4. Special Cases in Transportation Problems

(a) Unbalanced TP

- Supply > Demand: add a **dummy destination** with demand = excess supply; all costs in its column = 0
- Demand > Supply: add a **dummy source** with supply = shortage; all costs in its row = 0 (or shortage penalty)
- Then solve as balanced; units allotted to the dummy are actually **not shipped** (or not received)

Example: supplies 20, 30, 25 (total 75), demands 30, 25, 10 (total 65). Excess 10, so add a dummy D4 with demand 10 and zero costs.

	D1	D2	D3	D4*	Supply
S_1	8	6 **20**	10	0	20
S_2	9 **5**	7 **5**	4 **10**	0 **10**	30
S_3	3 **25**	4	2	0	25
Demand	30	25	10	10	

Optimal (VAM + MODI)
S1->D2 : 20
S2->D1:5, D2:5, D3:10
S2->dummy: 10 (unused)
S3->D1 : 25
Z = Rs 315

(b) Degenerate TP

*A TP is **degenerate** if the number of allocated cells is **less than m + n - 1** (it happens when a row and a column are exhausted at the same time). Remedy: put a very small quantity ε (≈ 0) in an **unallocated independent cell** (one that does not form a closed loop with the other allocations) and treat it as basic. At the end put ε = 0.*

	D_1	D_2	D_3	Supply
S_1	4 **10**	6 **ε**	8	10
S_2	5	3 **20**	7	20
S_3	9	2 **5**	5 **25**	30
Demand	10	25	25	

NWC: x_{11}=10 exhausts row 1 and column 1 together, only 4 cells < 3+3-1 = 5
add ε at (1,2)
then apply MODI as usual

Treat ε as a basic cell and apply MODI. Take $u_1 = 0$:

	D_1	D_2	D_3	Supply	u_i
S_1	4 **10**	6 **0**	8 Δ=-1	10	0
S_2	5 Δ=4	3 **20**	7 Δ=1	20	-3
S_3	9 Δ=9	2 **5**	5 **25**	30	-4
Demand	10	25	25		
v_j	4	6	9		

u=(0,-3,-4) v=(4,6,9)
Δ_{13} = 8-(0+9) = -1
ε cell leaves (θ = ε)
ε moves to (1,3)

After the exchange (θ = ε ≈ 0, so cost does not change) all Δ ≥ 0. Optimal cost = 10(4) + 20(3) + 5(2) + 25(5) = 40+60+10+125 = ***Rs 235****, with ε = 0.*

(c) Other variations

- *Maximisation TP (profit given): convert by subtracting every profit from the largest profit, or multiply by -1, then minimise*
- *Prohibited route: assign a very large cost M to that cell so that it never gets an allocation*
- *Multiple optimal solutions: an unallocated cell with $\Delta = 0$ in the optimal table*
- *Transhipment (intermediate points) and capacity-restricted routes are treated as extensions of the TP*

Rules for the closed loop (MODI)

- *Loop starts and ends at the entering (empty) cell; every other corner is an **allocated** cell*
- *Moves are only **horizontal or vertical**; each turn is at an allocated cell; alternate signs + - + - ...*
- *The loop may jump over (cross) other cells, but cannot turn there. For a given non-degenerate BFS the loop is **unique***

Maximisation TP - example idea

*Profit per unit: S1 -> (12, 18, 6), S2 -> (14, 3, 10). Convert: largest profit = 18, so new costs = 18 - profit = (6, 0, 12) and (4, 15, 8). Solve as a normal minimisation; the final Z is computed from the **original profits**.*

TP versus AP - short comparison

Point	Transportation	Assignment
Supply / demand	any positive numbers	exactly 1 each
Matrix	$m \times n$ (rectangular)	$n \times n$ (square)
x_{ij}	any non-negative	0 or 1 only
Basic cells	$m + n - 1$	n assignments
Method	NWC/LCM/VAM + MODI	Hungarian method

Applications

- *TP: distribution of goods from factories to depots, allocation of fuel/raw material, scheduling of production among plants*
- *AP: assigning workers to machines, salesmen to territories, cabs to customers, teachers to classes, contracts to bidders*

5. Assignment Problem (AP)

*The AP is a special TP in which n jobs are assigned to n workers (machines) on a **one-to-one basis** so that total cost (time) is minimum (or total profit is maximum). Each supply = each demand = **1**.*

Min $Z = \sum_i \sum_j c_{ij}\, x_{ij}$ *where* $x_{ij} = 1$ *if worker i is assigned job j, else 0*

$\sum_j x_{ij} = 1$ *(each worker one job)* $\sum_i x_{ij} = 1$ *(each job one worker)*

- Square cost matrix ($n \times n$). Number of assignments = n. TP methods work, but the **Hungarian method** is much quicker (degenerate TP otherwise)

Hungarian method - algorithm (Kuhn / Flood)

- 1. If the matrix is not square, add a dummy row/column of zeros. For maximisation, subtract all entries from the largest entry
- 2. **Row reduction:** subtract the smallest element of each row from all elements of that row
- 3. **Column reduction:** subtract the smallest element of each column from that column
- 4. Cover **all zeros** with the **minimum number of horizontal/vertical lines**
- 5. If lines = n -> optimal assignment exists: go to step 7. If lines < n go to step 6
- 6. Find the smallest **uncovered** element k. Subtract k from all uncovered elements, add k to elements at line intersections, leave others. Return to step 4
- 7. Assign: start with a row/column having a **single zero**, tick it, cross out other zeros of that row and column; repeat

Example: assign 4 jobs to 4 men, minimise cost (Rs)

Given matrix

	J1	J2	J3	J4
M1	15	15	23	25
M2	17	39	18	16
M3	17	29	24	6
M4	28	31	15	14

Step 2 - row reduction (row minima 15, 16, 6, 14 subtracted)

	J1	J2	J3	J4
M1	0	0	8	10
M2	1	23	2	0
M3	11	23	18	0
M4	14	17	1	0

Step 3 - column reduction (column minima 0, 0, 1, 0). Now cover all zeros: row M1, column J3 and column J4 -> only 3 lines < 4, so the assignment is not yet possible.

	J1	J2	J3	J4
M1	0	0	7	10
M2	1	23	1	0
M3	11	23	17	0
M4	14	17	0	0

Step 6 Smallest uncovered element = 1. Subtract 1 from uncovered cells (rows M2-M4, columns J1-J2), add 1 where lines cross (M1,J3) and (M1,J4).

	J1	J2	J3	J4
M1	0	0	8	11
M2	0	22	1	0
M3	10	22	17	0
M4	13	16	0	0

Now four lines are needed -> optimal. Assign: ***M3->J4*** *(only zero in row M3),* ***M4->J3****,* ***M2->J1****,* ***M1->J2****.*

Optimal assignment: *M1->J2 (15), M2->J1 (17), M3->J4 (6), M4->J3 (15).*
Minimum cost = 15 + 17 + 6 + 15 = Rs 53

Variations of the assignment problem

- *Maximisation: subtract each element from the maximum element of the matrix, then minimise*
- *Unbalanced (m ≠ n): add dummy rows/columns with zero cost. Job assigned to a dummy is left undone*
- *Restricted/prohibited assignment: put a very large cost M in that cell*
- *Multiple optimal solutions: when, in the final matrix, there are alternative sets of independent zeros*
- *Crew assignment / sales territories: typical practical uses; the Hungarian method works for any square cost matrix*

Maximisation example (profit matrix, Rs)

Profit of assigning worker W_i to job J_j:

	J1	J2	J3	J4
W1	16	10	14	11
W2	14	11	15	15
W3	15	15	13	12
W4	13	12	14	15

Largest element = 16. Subtract every element from 16 to get the loss (opportunity) matrix, then do row reduction (column minima are all 0):

	J1	J2	J3	J4
W1	0	6	2	5
W2	1	4	0	0
W3	0	0	2	3
W4	2	3	1	0

Four independent zeros exist: W1->J1, W2->J3, W3->J2, W4->J4.
Maximum profit = 16 + 15 + 15 + 15 = Rs 61 (read from the ORIGINAL profit matrix).

Unbalanced assignment (3 men, 4 jobs)

Add a **dummy man** (row of zeros) to make the matrix 4 × 4. After row reduction, three lines cover all zeros (< 4), smallest uncovered element = 2; adjusting gives:

	J1	J2	J3	J4
M1	0	0	7	1
M2	0	3	0	3
M3	1	1	0	0
Dummy	2	0	0	2

Assign M1->J1 (10), M2->J3 (7), M3->J4 (11), dummy->J2 (J2 stays undone). Cost = 10 + 7 + 11 = **Rs 28** for original matrix [10 12 19 11 / 5 10 7 8 / 12 14 13 11].

Restricted assignment

If worker i cannot do job i (diagonal prohibited), put M (very large) on the diagonal and solve normally. Example matrix [M 6 9 7 / 8 M 5 4 / 7 6 M 8 / 9 4 7 M] gives W1->J4, W2->J3, W3->J1, W4->J2 with total cost 7+5+7+4 = **23**.

6. Traveling Salesman Problem (TSP)

A salesman must **visit each of n cities exactly once** and return to the starting city so that the total distance (cost/time) is minimum. Its model is like an assignment problem with the extra condition that the assignment forms **one single tour**.

- Cost matrix c_{ij} (distance city i to city j); diagonal $c_{ii} = \infty$ (M), since a city cannot be followed by itself
- (n - 1)! possible tours for an asymmetric problem - too many to list when n is large

Solution procedure (assignment + sub-tour elimination)

- Solve as an AP with the Hungarian method ignoring the tour condition
- If the solution gives **one cycle through all cities** -> optimal tour
- If it splits into **sub-tours**, use branch-and-bound: forbid one arc of the sub-tour (cost M)

and re-solve, or patch the sub-tours using the least-cost link

Example: 5 cities A, B, C, D, E (distances)

	A	B	C	D	E
A	∞	3	6	2	3
B	3	∞	5	2	3
C	6	5	∞	6	4
D	2	2	6	∞	6
E	3	3	4	6	∞

Hungarian method on this matrix gives the assignment A->B, B->D, D->A and C->E, E->C with cost 3+2+2+4+4 = 15. This is ***not a tour****: it has two sub-tours A-B-D-A and C-E-C. So 15 is only a lower bound.*

Break the sub-tours by branching (forbid an arc such as D->A, re-solve, or link the cycles through the cheapest exchange). The best complete tour is:

A -> D -> B -> C -> E -> A *with distance 2 + 2 + 5 + 4 + 3 =* ***16 units****.*
(Verified by comparing all (5-1)! = 24 tours; no tour is shorter than 16.)

Feature	Assignment	TSP
Solution	any permutation	one single cycle (tour)
Extra condition	none	no sub-tours
Method	Hungarian	Hungarian + branch & bound

7. Transshipment Problem & Method Comparison

In a ***transshipment problem*** *goods may move from a source to a destination* ***through intermediate points*** *(other sources or destinations). Total cost = sum of costs along the chosen route.*

- *Treat every source and destination as both a possible origin and possible destination*
- *Buffer stock B = total supply (or total demand) is added to every supply and demand figure, so each point can hold flow in transit*
- *Cost from a point to itself = 0. Solve the enlarged balanced TP by VAM + MODI; flow on c_{ii} cells is just a balancing quantity*

Merits and demerits of the IBFS methods

Method	Merits	Demerits
NWCR	Very simple, no thinking	Ignores cost; far from optimum
LCM	Uses cost; quick; good start	Local choice, may miss better plan
VAM	Best start; fewest MODI steps	More calculation (penalties)

Common mistakes to avoid (exam tips)

- Forgetting to balance the problem first (dummy row/column with ZERO cost)
- Having fewer than $m + n - 1$ allocations and not adding ε; or adding ε to a cell that forms a loop
- Taking u, v from empty cells - they must be computed only from allocated cells
- In MODI, forgetting that θ is taken from the minus cells only, and the leaving cell has the smaller allocation
- In Hungarian method: covering zeros with more lines than the minimum; not adding k at intersections
- Maximisation AP: reading the final answer from the transformed (loss) matrix instead of the original profits
- TSP: leaving sub-tours in the answer - a valid tour visits each city exactly once

Worked mini-check: is a given plan optimal?

Given an allocation with $m + n - 1$ cells: (1) write $u_i + v_j = c_{ij}$ for each allocated cell, (2) compute Δ for empty cells, (3) if no $\Delta < 0$ the plan is optimal. If one $\Delta = 0$ there is an alternate optimum with the same cost, obtained by bringing that cell in through its loop.

8. Quick Revision & Important Questions

1. Give the mathematical form of a TP; explain NWC, LCM and VAM with an example.
2. MODI method steps; how to deal with unbalanced and degenerate TP.
3. Formulate the assignment problem; Hungarian method with maximisation variation.
4. Explain the TSP and its relation to the assignment problem.

Remember: basic cells = $m + n - 1$; $\Delta_{ij} = c_{ij} - u_i - v_j$; optimal if all $\Delta \geq 0$; VAM penalty = difference of two lowest costs; Hungarian: lines = n => stop.

Question bank (university pattern)

2-marks: define balanced TP; why dummy is added; what is degeneracy in TP; why Hungarian method needs a square matrix; define sub-tour.

5-marks: VAM with an example; steps of MODI; assignment of maximisation type; unbalanced assignment problem.

10-marks: solve a 3×4 or 4×4 TP with VAM + MODI; solve a 4×4 assignment problem by the Hungarian method; TSP as an assignment problem.

UNIT III

Game Theory & Job Sequencing

Handwritten study notes | B.Tech | Operations Research

Syllabus topics covered in this unit:

- Two-person zero-sum games: maximin-minimax principle, saddle point
- Games without saddle points: mixed strategies
- Dominance, arithmetic (oddments), algebraic and graphical methods
- Job sequencing: flow shop and job shop scheduling
- n jobs through 2 machines, n jobs through m machines, graphical method

1. Introduction to Game Theory

Game theory *is a mathematical study of decision making in situations of* ***conflict or competition****, where the outcome for one decision-maker depends on the decisions of the others (rivals). It was developed by* ***John von Neumann and Oskar Morgenstern*** *(1944, *Theory of Games and Economic Behaviour*).*

Important terms

- *Player: each competitor/decision maker (A, B). Two-person game = two players*
- *Strategy: complete plan of action for a player.* ***Pure strategy*** *- one fixed course always chosen;* ***mixed strategy*** *- a probability mix of pure strategies*
- *Pay-off matrix: table of gains/losses for each pair of strategies. By convention entries show payoff to the* ***row player A (maximiser)****; B (minimiser) pays*
- *Value of the game (V): expected payoff to A when both play optimally.* ***Fair game: V = 0.*** *Strictly determinable game: maximin = minimax*
- *Zero-sum game: gain of one player = loss of the other (sum of payoffs = 0)*
- *Optimal strategy: the strategy that gives a player the most favourable outcome regardless of what the opponent does*

Assumptions of a two-person zero-sum game

- *Each player has a finite number of strategies, known to both. Players act rationally and simultaneously*
- *Each player tries to maximise the minimum gain (A) or minimise the maximum loss (B); payoffs are known with certainty*

Types of games

Basis	Types
Number of players	Two-person, n-person
Sum of payoffs	Zero-sum, non-zero-sum (variable-sum)
Number of strategies	Finite, infinite
Strategy used	Pure-strategy, mixed-strategy
Co-operation	Co-operative, non-co-operative

2. Maximin - Minimax Principle and Saddle Point

- Player A (maximiser) finds the **minimum of each row**, then takes the **maximum** of those: maximin value $\underline{V} = \max_i \min_j a_{ij}$
- Player B (minimiser) finds the **maximum of each column**, then takes the **minimum** of those: minimax value $\overline{V} = \min_j \max_i a_{ij}$
- Always maximin ≤ minimax. If **maximin = minimax = V** the game has a saddle point: the cell is the **minimum of its row and maximum of its column**. Pure strategies are optimal and V is the value of the game

Example 1 - game with saddle point

A \ B	B1	B2	B3	Row min
A1	1	3	1	1
A2	0	-4	-3	-4
A3	1	5	-1	-1
Col max	1	5	1	

Maximin = max(1, -4, -1) = 1. Minimax = min(1, 5, 1) = 1. Equal -> saddle point exists. Saddle points: (A1, B1) and (A1, B3). **Optimal: A plays A1, B plays B1 or B3, Value V = 1** (game favours A).

Note: a game may have more than one saddle point; all have the same value. If V = 0 it is a **fair** game; if no saddle point exists we use **mixed strategies**.

3. Games without Saddle Point - Mixed Strategies

When maximin ≠ minimax, neither player can fix one pure strategy (the opponent could exploit it). Each player chooses strategies **at random with fixed probabilities**, so that the expected payoff is optimal. A plays strategies with probabilities $p_1, p_2, \ldots$ ($\sum p_i = 1$); B with $q_1, q_2, \ldots$ ($\sum q_j = 1$).

Minimax theorem (von Neumann): for every finite two-person zero-sum game there exists a value V and optimal (pure or mixed) strategies for both players such that *max-min of expected payoff = min-max of expected payoff = V*.

Formulae for a 2 × 2 game

Pay-off matrix [a b ; c d] (A's rows 1, 2; B's columns 1, 2) with no saddle point:

$p_1 = (d - c) / (a + d - b - c)$, $p_2 = 1 - p_1$

$q_1 = (d - b) / (a + d - b - c)$, $q_2 = 1 - q_1$

Value $V = (ad - bc) / (a + d - b - c)$

Example 2

A \ B	B1	B2	Row min
A1	2	4	2
A2	6	3	3
Col max	6	4	

Maximin = 3, minimax = 4 -> no saddle point. Here a=2, b=4, c=6, d=3, a+d-b-c = -5.

$p_1 = (3 - 6)/(-5) = 3/5$, $p_2 = 2/5$. $q_1 = (3 - 4)/(-5) = 1/5$, $q_2 = 4/5$.

$V = (2\cdot3 - 4\cdot6)/(-5) = (-18)/(-5) =$ **18/5 = 3.6**

Check: A's expected payoff vs B1 = 2(3/5)+6(2/5) = 3.6; vs B2 = 4(3/5)+3(2/5) = 3.6 (equal, hence optimal).

Arithmetic (oddments) method for 2 × 2

- Subtract the entries of **column 2 from column 1** (ignore sign) and write the result against the rows in reverse: these are the row **oddments**. Repeat with the rows for the column oddments
- Probability = oddment / sum of oddments

	B1	B2	Row oddment	Prob.
A1	2	4	|6-3| = 3	$p_1 = 3/5$
A2	6	3	|2-4| = 2	$p_2 = 2/5$
Col oddment	|4-3| = 1	|2-6| = 4		
Prob.	$q_1 = 1/5$	$q_2 = 4/5$		

Row oddments: A1 gets |c-d| = 3, A2 gets |a-b| = 2 (the difference of the OTHER row, taken

across). Column oddments: B1 gets |b-d| = 1, B2 gets |a-c| = 4. Same answer as the formula; V = (2·3 + 6·2)/5 = 3.6.

Algebraic method (equal expected payoffs)

Let A play A1 with probability p and A2 with (1-p). Against B1 and B2, expected payoffs must be equal: 2p + 6(1-p) = 4p + 3(1-p) => 6 - 4p = 3 + p => p = 3/5. Substitute to get V = 2(3/5)+6(2/5) = 3.6. In the same way B's q is found from 2q + 4(1-q) = 6q + 3(1-q) => q = 1/5.

4. Rules of Dominance

Dominance reduces the size of a payoff matrix by deleting strategies that will never be used by a rational player.

- *Row rule: if every element of row r is ≥ the corresponding element of row s, then row s (smaller) is dominated -> delete row s (A never plays it)*
- *Column rule: if every element of column r is ≤ the corresponding element of column s, then column s (larger) is dominated -> delete column s (B never plays it)*
- *Modified dominance: a row (column) may also be dominated by the **average / convex combination** of two or more other rows (columns)*
- *Dominance can be applied repeatedly. The probabilities of deleted strategies are zero*

Example 3 - solve using dominance

A \ B	B1	B2	B3
A1	1	7	2
A2	6	2	7
A3	5	1	6

- *A3 = (5, 1, 6) ≤ A2 = (6, 2, 7) in every column, so A2 dominates A3 -> delete A3*
- *Now B3 = (2, 7) ≥ B1 = (1, 6) in each row; B minimises, so B1 dominates B3 -> delete B3*

A \ B	B1	B2
A1	1	7
A2	6	2

2 × 2 game with a=1, b=7, c=6, d=2 (no saddle: maximin 2, minimax 6). a+d-b-c = -10.

$p_1 = (2 - 6)/(-10) = 2/5$, $p_2 = 3/5$; $q_1 = (2 - 7)/(-10) = 1/2$, $q_2 = 1/2$; $V = (2 - 42)/(-10) = 4$

Optimal: *A plays (A1, A2, A3) with (2/5, 3/5, 0); B plays (B1, B2, B3) with (1/2, 1/2, 0); Value = 4.*

5. Graphical Method for 2 × n or m × 2 Games

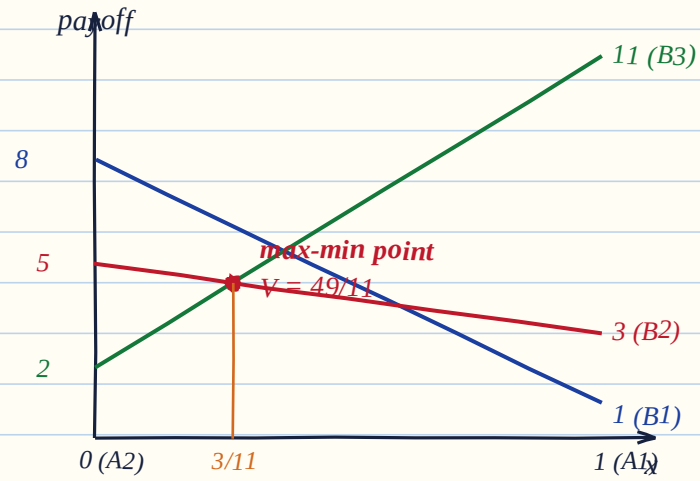
*Used when one player has only **two** strategies. Plot the expected payoff against each opposing pure strategy as a straight line; find the **highest point of the lower envelope** (for A) or the **lowest point of the upper envelope** (for B).*

- *Let A play A1 with probability x and A2 with 1 - x ($0 \le x \le 1$)*
- *For each column j of B draw $E_j(x) = a_{1j}x + a_{2j}(1 - x)$. A wants to **maximise the minimum** of these lines*
- *The point where the lower envelope is **highest** gives x* and V. The two lines meeting there identify B's active strategies; solve the 2 × 2 sub-game for q*

Example 4: 2 × 3 game

A \ B	B1	B2	B3
A1	1	3	11
A2	8	5	2

$E_1 = 8 - 7x$, $E_2 = 5 - 2x$, $E_3 = 2 + 9x$.



***Lower envelope** peaks where E_2 meets E_3 (B1 never used):*

$2 + 9x = 5 - 2x \Rightarrow x = 3/11$

$V = 2 + 27/11 = 49/11 \approx 4.45$

Sub-game B2, B3 gives B's mix:

$3q + 11(1-q) = 5q + 2(1-q)$

$\Rightarrow q = 9/11$ (B2), $2/11$ (B3)

***Solution:** A plays A1 with 3/11, A2 with 8/11. B plays B1 with 0, B2 with 9/11, B3 with 2/11. Value of game V = 49/11.*

Check A's rows: A1 -> 3(9/11) + 11(2/11) = 49/11; A2 -> 5(9/11) + 2(2/11) = 49/11 (equal, so optimal).

Other methods (names only)

- *Linear programming method: any m × n game can be converted to an LP; A's problem is the dual of B's problem (see Unit I duality)*
- *Matrix (inverse) method for square games; iteration/Brown's method for approximate solutions*

Linear programming formulation of a game (idea)

Make all payoffs positive (add a constant if needed). A plays x_i ($\sum x_i = 1$), V = value of game.

Player A: Max V s.t. $\sum_i a_{ij} x_i \geq V$ (for every column j), $\sum x_i = 1$, $x_i \geq 0$.

Put $y_i = x_i/V$: Min $Z = \sum y_i$ s.t. $\sum a_{ij} y_i \geq 1$, $y \geq 0$ and then $V = 1/Z$. B's problem is the DUAL of A's problem.

Applications and limitations of game theory

- Uses: pricing and advertising competition between firms, bidding and auctions, military strategy, labour-management bargaining, tender decisions
- Limits: assumes rival is rational and both know all payoffs; very large games are hard; real life seldom has exactly two players with strictly opposed interests

6. Job Sequencing - Introduction

Sequencing is the problem of finding the **order in which n jobs should be processed on m machines** so that a chosen measure (total elapsed time, idle time, cost) is optimised. Total possible sequences = $(n!)^m$ in general.

Terminology

- Processing time (t_{ij}): time job i needs on machine j (independent of the order)
- Processing order: the order of machines through which a job must pass (e.g. A -> B -> C)
- Total elapsed time (T): time from start of the first job on the first machine to completion of the last job on the last machine
- Idle time: time a machine is waiting with no job to process, within [0, T]
- No-passing rule: the same order of jobs is kept on every machine

Assumptions

- Machines are always available; one machine does one job at a time; a job is on only one machine at a time
- Once begun, a job is processed to completion (no pre-emption); transit time is negligible (included in t_{ij})

Types of scheduling problems

Type	Meaning
Flow shop	all jobs have the SAME machine order (assembly line, e.g. M1 -> M2)
Job shop	different jobs have DIFFERENT routings; more complex
n jobs, 2 machines	Johnson's rule - exact optimum
n jobs, 3 machines	Johnson's rule if conditions satisfied
n jobs, m machines	extension of Johnson's rule (special conditions)
2 jobs, m machines	graphical method

7. Processing n Jobs through Two Machines (Johnson's Rule)

Johnson's algorithm (minimises total elapsed time for jobs going M1 -> M2):

1. *List the processing times of all jobs on M1 and M2.*
2. *Find the **smallest** time in the whole list.*
3. *If it is on **M1**, place that job at the **first** free position of the sequence; if it is on **M2**, place it at the **last** free position.*
4. *Delete that job; repeat steps 2-3 for the remaining jobs. Ties: pick either (several optimal sequences possible).*

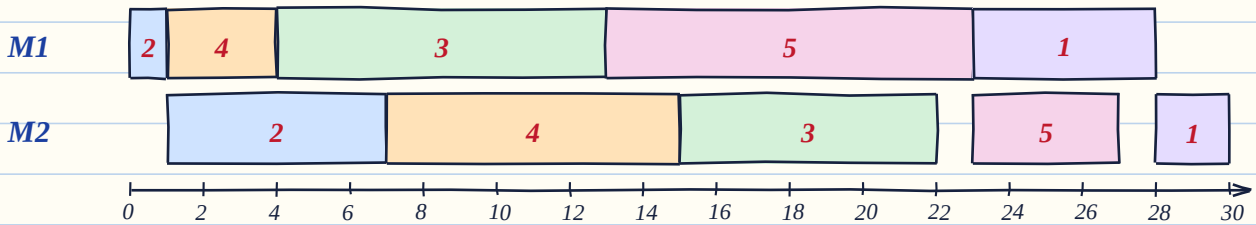
Example 5: five jobs, two machines (hours)

Job	1	2	3	4	5
M1	5	1	9	3	10
M2	2	6	7	8	4

- Smallest = 1 (Job 2 on M1) -> first position: 2 _ _ _ _
- Next smallest = 2 (Job 1 on M2) -> last position: 2 _ _ _ 1
- Next = 3 (Job 4 on M1) -> next first: 2 4 _ _ 1. Next = 4 (Job 5 on M2) -> last free: 2 4 _ 5 1. Remaining Job 3 -> middle

Optimal sequence: 2 - 4 - 3 - 5 - 1

Job	M1 in	M1 out	M2 in	M2 out
2	0	1	1	7
4	1	4	7	15
3	4	13	15	22
5	13	23	23	27
1	23	28	28	30



Total elapsed time = 30 hours.

Idle time on M1 = 30 - (5+1+9+3+10) = 2 hrs (Job 1 leaves M1 at 28). Idle time on M2 = 3 hrs = 1 (initial wait) + 1 (between Jobs 3 and 5) + 1 (between Jobs 5 and 1).

Formula: Idle time on M2 = ∑ (start of job on M2 - finish of previous job on M2) + initial wait (= $t_{1,M1}$); Idle time on M1 = T - ∑ $t_{i,M1}$.

8. Processing n Jobs through Three Machines

*Johnson's rule extends to 3 machines **only if** at least one of the following conditions holds:*

> *(i) Minimum time on M1 ≥ Maximum time on M2 or (ii) Minimum time on M3 ≥ Maximum time on M2*
> *Then replace the 3 machines by two **fictitious machines**: G = M1 + M2 and H = M2 + M3, and apply Johnson's rule to (G, H).*

Example 6: seven jobs, machines A, B, C

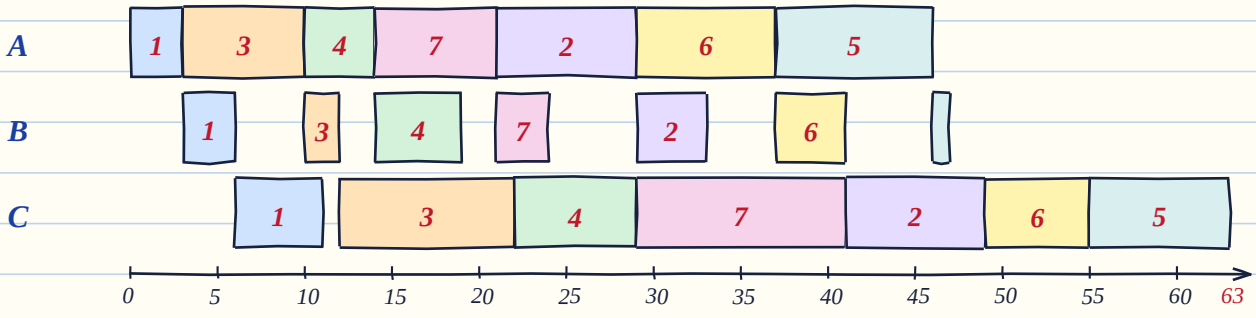
Job	1	2	3	4	5	6	7
A	3	8	7	4	9	8	7
B	3	4	2	5	1	4	3
C	5	8	10	7	8	6	12

Min A = 3, Max B = 5, Min C = 5. Condition (ii): min C = 5 ≥ max B = 5 is satisfied. Fictitious machines:

Job	1	2	3	4	5	6	7
G=A+B	6	12	9	9	10	12	10
H=B+C	8	12	12	12	9	10	15

*Johnson's rule on (G, H) gives the optimal sequence **1 - 3 - 4 - 7 - 2 - 6 - 5**. Now find the in-out times on the **original** machines A, B, C:*

Job	A in-out	B in-out	C in-out
1	0 - 3	3 - 6	6 - 11
3	3 - 10	10 - 12	12 - 22
4	10 - 14	14 - 19	22 - 29
7	14 - 21	21 - 24	29 - 41
2	21 - 29	29 - 33	41 - 49
6	29 - 37	37 - 41	49 - 55
5	37 - 46	46 - 47	55 - 63



Minimum total elapsed time = 63 hours.
Idle time: machine A = 17 hrs, machine B = 41 hrs, machine C = 7 hrs (T - total processing time on that machine).

9. Processing n Jobs through m Machines

*For m > 3 machines the same idea is used. If **either** condition below holds, replace the m machines by two fictitious machines G and H and use Johnson's rule.*

Condition: $\min t_{i1} \geq \max t_{ij}$ *or* $\min t_{im} \geq \max t_{ij}$ *for* $j = 2, 3, \ldots, m-1$ *(all middle machines)*
$G_i = t_{i1} + t_{i2} + \ldots + t_{i,m-1}$ $H_i = t_{i2} + t_{i3} + \ldots + t_{im}$

Example 7: four jobs, four machines

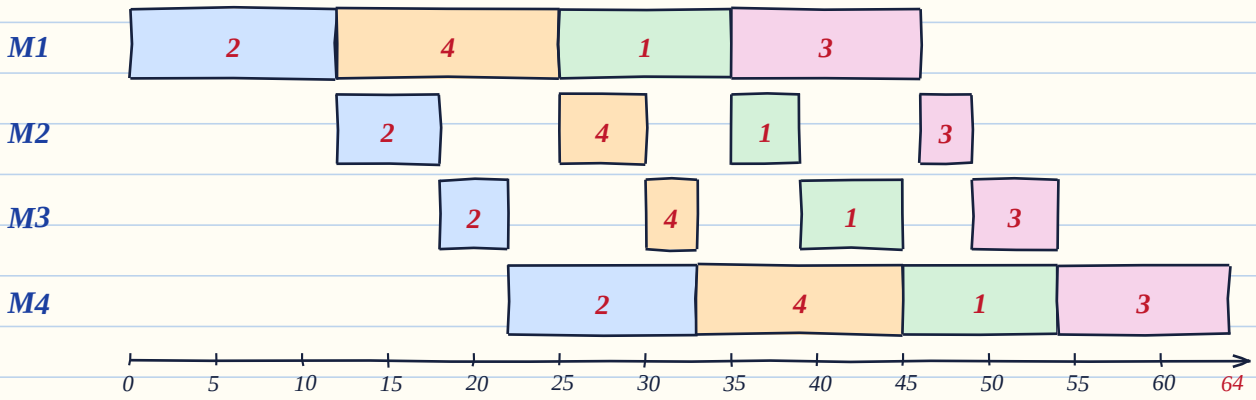
Job	1	2	3	4
M1	10	12	11	13
M2	4	6	3	5
M3	6	4	5	3
M4	9	11	10	12

Min M1 = 10 ≥ Max(M2) = 6 and Max(M3) = 6, so the condition holds. G = M1+M2+M3, H = M2+M3+M4.

Job	1	2	3	4
G	20	22	19	21
H	19	21	18	20

Smallest value = 18 (Job 3 on H) -> last; next 19 (Job 1 on H) -> second last; 20 (Job 4 on H) -> next; then Job 2. Sequence = ***2 - 4 - 1 - 3****.*

Job	M1	M2	M3	M4
2	0-12	12-18	18-22	22-33
4	12-25	25-30	30-33	33-45
1	25-35	35-39	39-45	45-54
3	35-46	46-49	49-54	54-64



Total elapsed time = 64 hrs. *Idle times: M1 = 18, M2 = 46, M3 = 46, M4 = 22 hrs.*

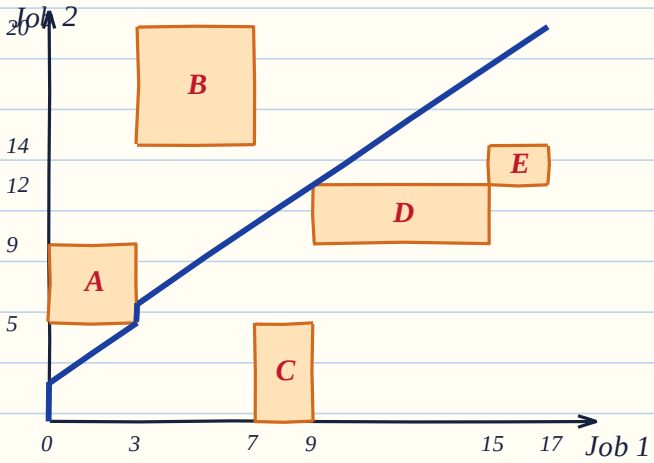
10. Two Jobs through m Machines - Graphical Method

Two jobs, each to be processed on the ***same m machines but in different orders****. A graph is used to find the minimum elapsed time.*

- *Mark Job 1's processing times along the* ***x-axis*** *and Job 2's along the* ***y-axis****, in their respective machine orders*
- *For each machine draw the* ***rectangle (block)*** *formed where both jobs need that machine - the two jobs cannot use it together, so the path cannot pass through the block*
- *From origin O to the end point draw a path using only horizontal (Job 1 works, Job 2 waits), vertical (Job 2 works) and 45° diagonal (both work) segments, avoiding blocks*
- *Elapsed time = horizontal length + vertical length + length of the diagonal pieces measured along one axis. Choose the path with the* ***fewest non-diagonal moves*** *(diagonals save time)*

Example 8

Job 1 sequence	A	B	C	D	E
Time	3	4	2	6	2
Job 2 sequence	C	A	D	E	B
Time	5	4	3	2	6



Optimal path (blue):

O -> (0,2): vertical (Job 2 alone, 2 hrs)
-> (3,5): diagonal (both work, 3 hrs)
-> (3,6): vertical (1 hr)
-> (17,20): diagonal (14 hrs)

Elapsed time = 2+3+1+14 = 20 hrs

= total time of Job 2 (20),
Job 1 waits 2 + 1 = 3 hrs.

Result: *minimum total elapsed time = 20 hours. Idle time for Job 1 = 20 - 17 = 3 hrs; for Job 2 = 0.*

Reading the graph: a diagonal step cannot enter a shaded block because both jobs would need the same machine at the same time.

11. Job Shop Scheduling & Dispatching Rules

*In a **job shop** every job may need a different set of machines in a different order, and many jobs compete for the same machine. Exact optimal solutions are too hard (up to $(n!)^m$ schedules), so shops use **priority (dispatching) rules** to choose which waiting job a free machine should process next.*

Rule	Meaning	Effect
FCFS	First come, first served	Fair; poor flow time
SPT	Shortest processing time first	Minimises mean flow time
LPT	Longest processing time first	Good machine balance
EDD	Earliest due date first	Reduces maximum lateness
Slack	Due date - now - remaining work (least first)	Protects urgent jobs
CR	Critical ratio = time left to due date / work left	CR < 1 means job is late

Performance measures

- *Makespan = completion time of the last job (total elapsed time); Flow time F_i = completion time - arrival time; Lateness L_i = completion - due date; Tardiness = max(0, L_i)*
- *Other measures: machine utilisation, work-in-process inventory, number of tardy jobs*

Example - single machine, 4 jobs (processing times 4, 2, 7, 3 days)

Rule	Order	Completion times	Mean flow time
FCFS	J1 J2 J3 J4	4, 6, 13, 16	39/4 = 9.75
SPT	J2 J4 J1 J3	2, 5, 9, 16	32/4 = 8.0

SPT gives the smaller mean flow time (8 days). This is the standard result: on one machine, **SPT minimises mean flow time**, and **EDD minimises maximum lateness**.

Flow shop vs Job shop

Point	Flow shop	Job shop
Routing	same order for all jobs	different for each job
Layout	product (assembly line)	process (functional)
Volume / variety	high volume, low variety	low volume, high variety
Number of schedules	n! (permutation)	$(n!)^m$
Technique	Johnson's rule etc.	dispatching rules, heuristics

12. Quick Revision & Important Questions

Game theory: maximin ≤ minimax; saddle point if equal; 2×2 formulae p_1 = (d-c)/(a+d-b-c), V = (ad-bc)/(a+d-b-c); dominance rows (delete smaller), columns (delete larger).

Sequencing: Johnson: smallest on M1 -> first, smallest on M2 -> last; 3 machines need min M1 ≥ max M2 or min M3 ≥ max M2; G = M1+M2, H = M2+M3; Idle time on last machine...

Likely questions: (1) Define a two-person zero-sum game, explain maximin-minimax principle. (2) Solve a 2×2 / 3×3 game with dominance. (3) Graphical solution of 2×n game.
(4) Johnson's rule: n jobs 2 machines with Gantt chart & idle time. (5) n jobs 3 machines. (6) 2 jobs m machines graphical method. (7) Differences between flow shop and job shop.

UNIT IV

Queuing Theory & Inventory Control

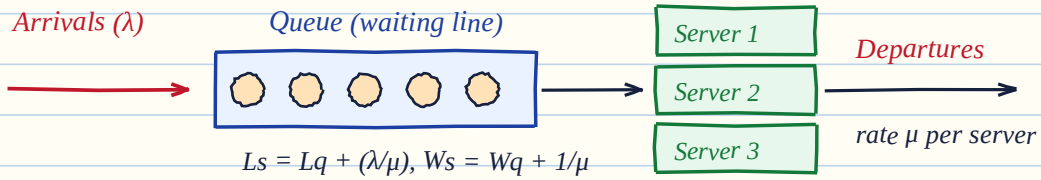
Handwritten study notes | B.Tech | Operations Research

Syllabus topics covered in this unit:

- Queuing: terminology, arrival and service patterns, birth-death process
- Single channel models (M/M/1) with infinite and finite queue length
- Multichannel model (M/M/C) with infinite queue length
- Inventory: EOQ with and without shortages, production model
- Buffer stock, quantity discount (single price break), selective control

1. Queuing Theory - Introduction

A **queue (waiting line)** forms when customers arriving for service must wait because the service facility is busy. Queuing theory builds mathematical models of arrivals, waiting and service in order to **balance the cost of waiting against the cost of providing service**. Examples: bank counters, hospital OPD, toll gates, ATM, airport runway, machines waiting for repair.



Elements / terminology of a queuing system

- *Input (arrival) pattern:* how customers arrive - usually **Poisson** with mean rate λ per unit time, i.e. inter-arrival times are exponential. May be single or bulk, finite or infinite
- *Service channel (mechanism):* number of servers and their arrangement - single channel, multi-channel in parallel, or in series (phases)
- *Service (departure) pattern:* time taken to serve a customer - usually **exponential** with mean service rate μ per unit time (mean service time 1/μ)
- *Population (calling source):* infinite (large) or finite source of customers
- *Queue discipline:* rule for selecting the next customer - **FIFO/FCFS**, LIFO, SIRO (random), priority (PR), pre-emptive priority
- *Customer behaviour:* balking (refuses to join a long queue), reneging (leaves after waiting), jockeying (switches queues), collusion
- *Capacity of system:* infinite or finite (maximum N in system); *steady state:* probabilities do not change with time (needs $\lambda < c\mu$)

Notation

Symbol	Meaning	Symbol	Meaning
λ	mean arrival rate	ρ	traffic intensity λ/μ (or $\lambda/c\mu$)
μ	mean service rate	P_n	probability of n in system
L_s	mean number in system	L_q	mean number in queue
W_s	mean time in system	W_q	mean waiting time in queue
c	number of servers	N	system capacity

Kendall's notation: $(a / b / c) : (d / e)$ a = arrival distribution, b = service distribution (M = Markovian/Poisson-exponential, D = deterministic, G = general), c = number of servers, d = system capacity, e = queue discipline. Example $(M/M/1):(\infty/FIFO)$.

Little's formulae: $L_s = \lambda W_s$, $L_q = \lambda W_q$, $W_s = W_q + 1/\mu$, $L_s = L_q + \lambda/\mu$.

2. Birth and Death Process

Arrival = **birth** (system size n -> n+1 with rate λ_n); departure = **death** (n -> n-1 with rate μ_n). In steady state the rate of flow **into** a state equals the rate **out of** it (balance equations).

State 0: $\lambda_0 P_0 = \mu_1 P_1$

State n ($n \geq 1$): $\lambda_{n-1} P_{n-1} + \mu_{n+1} P_{n+1} = (\lambda_n + \mu_n) P_n$

Solving: $P_n = [\, \lambda_0 \lambda_1 \dots \lambda_{n-1} / (\mu_1 \mu_2 \dots \mu_n) \,] P_0$, and $\sum P_n = 1$ gives P_0.

For constant rates $\lambda_n = \lambda$, $\mu_n = \mu$: $P_n = (\lambda/\mu)^n P_0 = \rho^n P_0$

Summing the geometric series: $P_0 = 1 - \rho$ (for $\rho < 1$) hence $P_n = (1 - \rho)\rho^n$. All the standard queue models are derived from this.

3. Model I: (M/M/1):(∞/FIFO) - Single Server, Infinite Queue

Conditions: Poisson arrivals (λ), exponential service (μ), one server, infinite capacity, FIFO, $\rho = \lambda/\mu < 1$.

$P_0 = 1 - \rho$; $P_n = (1 - \rho)\rho^n$; $P(n \geq k) = \rho^k$

$L_s = \rho/(1 - \rho) = \lambda/(\mu - \lambda)$ $L_q = \rho^2/(1 - \rho) = \lambda^2/[\mu(\mu - \lambda)]$

$W_s = 1/(\mu - \lambda)$ $W_q = \lambda/[\mu(\mu - \lambda)]$ $P(W_s > t) = e^{-(\mu-\lambda)t}$ L_n (non-empty queue) $= \mu/(\mu - \lambda)$

Example 1

Customers arrive at a service counter at 6 per hour (Poisson); service is exponential at 8 per hour. Find the performance measures.

Measure	Formula	Value
Traffic intensity ρ	λ/μ	$6/8 = 0.75$
P(system empty) P_0	$1 - \rho$	0.25
Mean number in system L_s	$\lambda/(\mu-\lambda)$	6/2 = 3 customers
Mean queue length L_q	$\rho^2/(1-\rho)$	0.5625/0.25 = 2.25
Time in system W_s	$1/(\mu-\lambda)$	0.5 hr = 30 min
Waiting time W_q	$\lambda/[\mu(\mu-\lambda)]$	6/16 = 0.375 hr = 22.5 min
P(3 or more in system)	ρ^3	0.4219
P(time in system > 30 min)	$e^{-(\mu-\lambda)(0.5)}$	$e^{-1} = 0.368$

Check by Little's law: $L_s = \lambda W_s = 6 \times 0.5 = 3$ and $L_q = \lambda W_q = 6 \times 0.375 = 2.25$ (agrees). The server is busy 75% of the time ($= \rho$).

Derivation of L_s and L_q

$L_s = \sum n P_n = (1 - \rho) \sum n \rho^n = (1 - \rho)\, \rho/(1 - \rho)^2 = \rho/(1 - \rho)$ (using $\sum n\rho^n = \rho/(1-\rho)^2$).

$L_q = \sum_{n\geq 1} (n - 1) P_n = L_s - (1 - P_0) = \rho/(1-\rho) - \rho = \rho^2/(1 - \rho)$.

Waiting time in system is exponential with mean $1/(\mu - \lambda)$, hence $W_s = L_s/\lambda = 1/(\mu - \lambda)$.

Example 1(b): design question

$\lambda = 10$ per hour, $\mu = 12$ per hour: $\rho = 0.833$, $L_s = 5$, $L_q = 4.17$, $W_s = 0.5$ hr, $W_q = 0.417$ hr.

What μ is needed so that the mean time in the system is at most 15 min? $W_s = 1/(\mu - 10) \leq 0.25$ hr $\Rightarrow$ $\mu - 10 \geq 4$ $\Rightarrow$ $\mu \geq 14$ per hour.

4. Model II: (M/M/1):(N/FIFO) - Finite Queue Length

At most **N customers** can be in the system (including the one in service); an arrival finding N present is turned away (**lost**). The system is stable for any ρ, even $\rho \geq 1$.

$P_0 = (1 - \rho)/(1 - \rho^{N+1})$ and $P_n = (1 - \rho)\, \rho^n/(1 - \rho^{N+1})$, $n = 0, 1, \ldots, N$ ($\rho \neq 1$; if $\rho = 1$, $P_n = 1/(N+1)$)

$L_s = \rho/(1-\rho) - (N+1)\rho^{N+1}/(1 - \rho^{N+1})$ $\quad L_q = L_s - (1 - P_0)$

Effective arrival rate $\lambda' = \lambda(1 - P_N)$ (P_N = probability an arrival is lost)

$W_s = L_s/\lambda'$, $\quad W_q = L_q/\lambda' = W_s - 1/\mu$

Example 2

$\lambda = 3$ per hour, $\mu = 4$ per hour and the room holds at most $N = 4$ customers. $\rho = 0.75$, $\rho^5 = 0.2373$.

Quantity	Working	Result
P_0	(1-0.75)/(1-0.2373)	0.3278
$P_4 = P(\text{lost})$	0.3278×0.75^4	0.1037 (10.4%)
L_s	3 - 5(0.2373)/0.7627	1.444
Effective λ'	3(1 - 0.1037)	2.689 per hr
L_q	1.444 - (1 - 0.3278)	0.772
W_s	1.444/2.689	0.537 hr (32.2 min)
W_q	0.772/2.689	0.287 hr (17.2 min)

5. Model III: (M/M/C):(∞/FIFO) - Multichannel

c identical servers in parallel, one common queue, each server has rate μ. Overall service rate = $n\mu$ when $n \le c$ busy, and $c\mu$ when $n \ge c$. Steady state needs $\rho = \lambda/(c\mu) < 1$.

$$P_0 = \left[\sum_{n=0}^{c-1} (\lambda/\mu)^n/n! \; + \; (\lambda/\mu)^c / (c! \, (1 - \lambda/c\mu)) \right]^{-1}$$

$$P_n = (\lambda/\mu)^n P_0/n! \;\; (n < c); \quad P_n = (\lambda/\mu)^n P_0/(c! \, c^{n-c}) \;\; (n \ge c)$$

$$L_q = P_0 (\lambda/\mu)^c \rho / [c! \, (1 - \rho)^2], \quad L_s = L_q + \lambda/\mu$$

$$W_q = L_q/\lambda, \quad W_s = W_q + 1/\mu = L_s/\lambda. \quad P(\text{wait}) = (\lambda/\mu)^c P_0 / [c!(1 - \rho)]$$

Example 3

Arrival rate $\lambda = 12$ per hour, each of $c = 2$ servers serves $\mu = 8$ per hour. Then $\lambda/\mu = 1.5$ and $\rho = 12/16 = 0.75$.

Quantity	Working	Result
P_0	$[1 + 1.5 + (1.5)^2/(2 \times 0.25)]^{-1} = [1 + 1.5 + 4.5]^{-1}$	1/7 = 0.1429
P(customer waits)	(2.25/(2×0.25)) × (1/7) = 4.5/7	0.643
L_q	(1/7)(2.25)(0.75)/[2(0.0625)]	1.929
L_s	1.929 + 1.5	3.429
W_q	1.929/12	0.161 hr = 9.6 min
W_s	0.161 + 1/8	0.286 hr = 17.1 min

Example 3(b): three servers

$\lambda = 18$ per hour, $\mu = 8$ per hour per server, $c = 3$: $\lambda/\mu = 2.25$, $\rho = 0.75$. $P_0 = [1 + 2.25 + 2.531 + 7.594]^{-1} = 1/13.375 = 0.0748$.

L_q	L_s	W_q	W_s
1.703	3.953	0.0946 hr (5.7 min)	0.2196 hr (13.2 min)

Compare Example 3 (2 servers, $\lambda = 12$): adding a server to handle more load keeps $\rho = 0.75$ but

a pooled queue gives shorter waits than separate single-server queues of the same load.

Summary of the three models

Model	Notation	Key formula
I	(M/M/1):(∞/FIFO)	$L_s = \lambda/(\mu-\lambda)$
II	(M/M/1):(N/FIFO)	$P_n = (1-\rho)\rho^n/(1-\rho^{N+1})$; $\lambda' = \lambda(1-P_N)$
III	(M/M/c):(∞/FIFO)	$L_q = P_0(\lambda/\mu)^c\rho/[c!(1-\rho)^2]$

Cost model: total cost per hour = (cost per server × c) + (waiting cost per customer-hour × L_s). Choose c that minimises it.

6. Inventory Control - Introduction

Inventory is the stock of materials (raw material, work-in-process, finished goods, spares) held to meet future demand. Inventory control decides **how much to order and when**, so that total cost is minimum while service is not interrupted.

Why hold inventory?

- To avoid stock-outs, to take advantage of quantity discounts, to smooth production, to guard against price rises and delays in delivery

Inventory costs

Cost	Symbol	Meaning
Holding (carrying) cost	C_h or C_1	per unit per year: interest, storage, insurance, obsolescence (often 15-30% of price)
Ordering (set-up) cost	C_0 or C_3	per order / per production run: paper work, transport, set-up
Shortage cost	C_s or C_2	per unit short per year: lost sales, penalty, goodwill
Purchase (production) cost	P	price per unit; matters only if price varies with Q

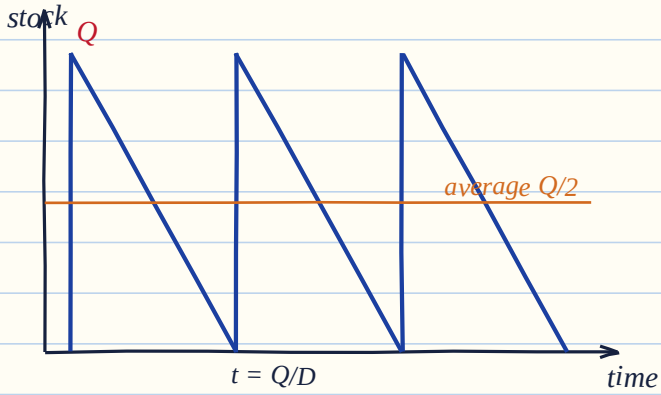
Terms

- D = annual demand; Q = order (lot) size; t = cycle time = Q/D; lead time L = delay between order and receipt
- Reorder level = demand during lead time + buffer stock; Buffer (safety) stock = protection against variation in demand/lead time

Deterministic models assume demand D and lead time are known and constant. Total annual cost = purchase + ordering + holding (+ shortage).

7. EOQ Model I - Economic Order Quantity (no shortages)

Assumptions: constant known demand rate, ***instantaneous replenishment****, no shortages, lead time zero/constant, price constant.*

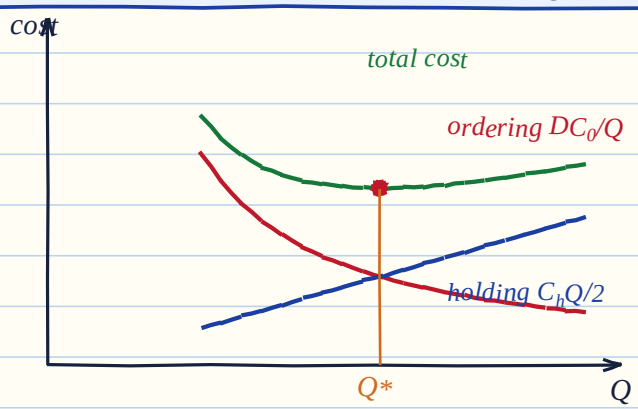


Annual ordering cost = $(D/Q)\,C_0$; annual holding cost = $(Q/2)\,C_h$. $TC(Q) = DP + DC_0/Q + C_hQ/2$

$dTC/dQ = 0$ gives $Q^* = \sqrt{(2DC_0/C_h)}$, $TC_{min} = DP + \sqrt{(2DC_0C_h)}$

Number of orders $N^* = D/Q^*$; cycle time $t^* = Q^*/D$. At the optimum ***ordering cost = holding cost****.*

Second derivative $= 2DC_0/Q^3 > 0$, so Q^* gives a minimum.



Example 4

$D = 2400$ units/year, $C_0 = Rs\ 100$ per order, $C_h = Rs\ 6$ per unit per year.

$Q^* = \sqrt{(2 \times 2400 \times 100/6)} = \sqrt{80000} = 283$ units (282.84)

Annual cost (excluding purchase) $= \sqrt{(2 \times 2400 \times 100 \times 6)} = \sqrt{2880000} = Rs\ 1697$ (ordering 848.5 + holding 848.5)

Orders per year $= 2400/282.84 = 8.49$; cycle time $= 0.1179$ yr ≈ 43 days.

Example 4(b): with purchase price

$D = 9000$ units/yr, price Rs 20, holding cost 20% of price ($C_h = Rs\ 4$), $C_0 = Rs\ 150$. $Q^* = \sqrt{(2 \times 9000 \times 150/4)} = \sqrt{675000} = 821.6$. Orders/yr = 10.95; cycle = 33 days. Annual cost $= 9000 \times 20 + \sqrt{(2 \times 9000 \times 150 \times 4)} = 180000 + 3286.3 = Rs\ 1,83,286$.

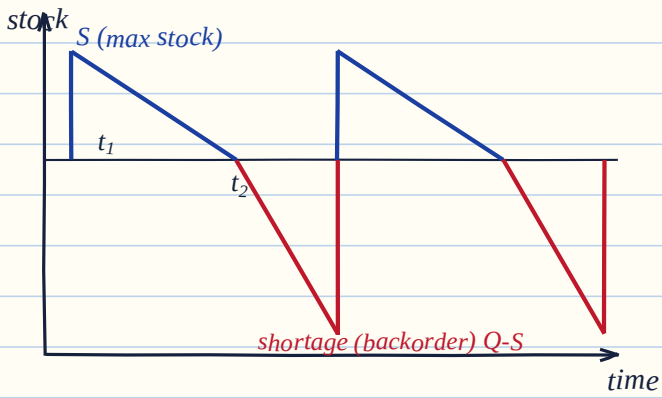
Assumptions and limitations of EOQ

- Demand constant and known; price independent of Q; whole lot arrives together; no stock-outs; single item, no capacity limit
- TC curve is **flat** near Q*: a 20% error in Q changes TC by only about 2% - EOQ is robust

Model	Demand	Replenishment	Shortages
EOQ I	constant	instantaneous	not allowed
EOQ II	constant	instantaneous	back-ordered
Production	constant	finite rate K	not allowed
Probabilistic	random	any	buffer stock used

8. EOQ Model II - With Shortages (back-ordering)

Shortages are allowed and **fully back-ordered**; the backlog is filled as soon as the next lot arrives. Shortage cost C_s per unit per year.



$Q^* = \sqrt{[\ 2DC_0(C_h + C_s) / (C_h\ C_s)\ ]} = EOQ \times \sqrt{[(C_h + C_s)/C_s]}$

Maximum inventory $S^* = Q^*\ C_s/(C_h + C_s)$, maximum shortage $Q^* - S^* = Q^*\ C_h/(C_h + C_s)$

$TC_{min} = \sqrt{[\ 2DC_0C_hC_s/(C_h + C_s)\ ]}$ $t_1 = S^*/D$ (stock period), $t_2 = (Q^*-S^*)/D$ (shortage period)

Example 5: same data as Example 4 with C_s = Rs 12 per unit per year.

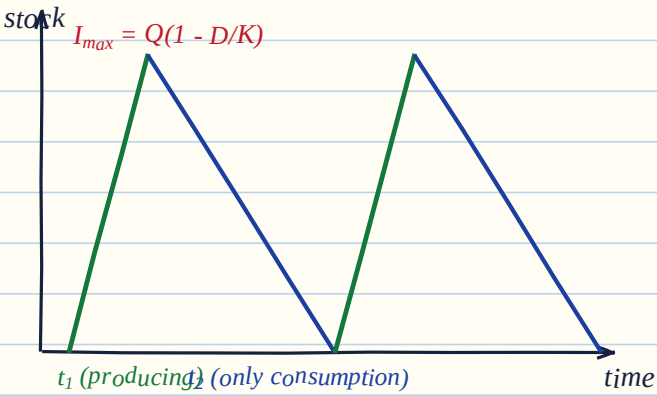
$Q^* = \sqrt{(2 \times 2400 \times 100 \times 18/72)} = \sqrt{120000} = 346.4$ units; $S^* = 346.4 \times 12/18 = 230.9$; maximum shortage = 115.5

$TC = \sqrt{(2 \times 2400 \times 100 \times 6 \times 12/18)} =$ Rs 1385.6 < 1697 (cheaper than the no-shortage plan).

As $C_s \to \infty$ the formula reduces to Model I (no shortages are worth allowing).

9. Production (Manufacturing) Model - finite replenishment

Items are produced in lots at rate K per year ($K > D$) and used at rate D at the same time. Stock builds up at $(K - D)$ during production time $t_1 = Q/K$, then falls at D.



$Q^* = \sqrt{[\,2DC_0 / (\,C_h(1 - D/K)\,)\,]}$ $\quad I_{max} = Q^*(1 - D/K)$ $\quad TC_{min} = \sqrt{[\,2DC_0C_h(1 - D/K)\,]}$

$t_1 = Q^*/K$ (production), cycle time $t = Q^*/D$, $t_2 = t - t_1$. As $K \to \infty$ the model becomes EOQ Model I.

Example 6: $D = 2400$, $K = 4800$, $C_0 = $ Rs 100, $C_h = $ Rs 6. $1 - D/K = 0.5$.

$Q^* = \sqrt{(2 \times 2400 \times 100/(6 \times 0.5))} = \sqrt{160000} =$ 400 units; $I_{max} = 400 \times 0.5 =$ 200;

$TC = \sqrt{(2 \times 2400 \times 100 \times 6 \times 0.5)} =$ Rs 1200; production time = 400/4800 = 1/12 yr (30 days); cycle = 1/6 yr (60 days).

Model with shortages and production: $Q^* = \sqrt{[2DC_0(C_h+C_s)/(C_hC_s(1-D/K))]}$ (extension for exam awareness).

10. Buffer Stock and Reorder Level

Real demand and lead time vary, so a **buffer (safety) stock** is held to reduce the chance of a stock-out.

Reorder level (ROL) = (average demand × lead time) + buffer stock

Buffer stock = (maximum lead-time demand) - (average lead-time demand)

ROL (max-based) = maximum usage rate × maximum lead time. Minimum level = ROL - (normal usage × normal lead time)

Maximum level = ROL + ROQ - (minimum usage × minimum lead time). Average stock = minimum level + ROQ/2

Example: normal use 20 units/day, lead time 10 days; maximum use 30/day, lead time up to 12 days. ROL = 30 × 12 = 360; average lead-time demand = 200, so buffer stock = 360 - 200 = 160 units. If lead-time demand is normal with mean μ_L and s.d. σ_L: buffer = $z\,\sigma_L$ for a service level ($z = 1.65$ for 95%).

11. Quantity Discount Model - Single Price Break

The supplier offers a **lower price p_2** if $Q \geq q_b$(break quantity), else price p_1. Holding cost is usually a percentage i of price ($C_h = i\,p$). Total cost $TC = Dp + DC_0/Q + i\,p\,Q/2$.

Step 1: compute EOQ at the lowest price p_2. If $EOQ_2 \geq q_b$: it is feasible and OPTIMAL (stop).
Step 2: otherwise compute EOQ_1 at p_1 (feasible if $< q_b$) and its total cost TC_1.
Step 3: compute TC at $Q = q_b$ with price p_2. Choose the smaller total cost.

Example 7(a): D = 1000, C_0 = Rs 100, i = 20%; price Rs 50 for Q < 100, Rs 45 for Q ≥ 100

EOQ at Rs 45: $C_h = 9$, $Q = \sqrt{(2 \times 1000 \times 100/9)} = 149.07 \geq 100$ -> feasible, so $Q^* = 149$ units; $TC = 45000 + 670.8 + 670.8 =$ Rs 46,341.6.

Example 7(b): D = 1000, C_0 = Rs 50, i = 20%; price Rs 20 for Q < 500, Rs 18 for Q ≥ 500

Option	Q	Price	$TC = Dp + DC_0/Q + C_hQ/2$
EOQ at Rs 18 ($C_h = 3.6$)	166.7 < 500	-	not feasible
EOQ at Rs 20 ($C_h = 4$)	158.1	20	20000 + 316.2 + 316.2 = 20,632.5
Break quantity	500	18	18000 + 100 + 900 = 19,000

Minimum cost is at Q = 500 (Rs 19,000), so accept the discount and order the break quantity of 500.

12. Selective Inventory Control

Treating all items alike is wasteful. **Selective control** classifies items so that management effort is proportional to importance.

Technique	Basis of classification	Categories
ABC	annual consumption value (Pareto 80:20)	A, B, C
VED	criticality of item to production	Vital, Essential, Desirable
FSN	rate of consumption / movement	Fast, Slow, Non-moving
HML	unit price	High, Medium, Low
SDE	difficulty of procurement	Scarce, Difficult, Easy
XYZ	value of stock held	X (high), Y, Z (low)

ABC analysis - procedure

- Find annual usage value = annual demand × unit price for every item; arrange in **descending** order
- Compute cumulative value % and cumulative item %; draw the cumulative (Lorenz-type) curve; fix A/B/C cut-offs

Class	% of items	% of value	Control
A	10-20%	70-80%	tight: close records, frequent review, small safety stock
B	~30%	15-20%	moderate: periodic review
C	50-60%	5-10%	loose: bulk orders, simple records, large stock

Example 8

Item	Value (Rs '000)	Cum. value %	Cum. items %	Class
I1	450	45.0	12.5	A
I2	280	73.0	25.0	A
I3	120	85.0	37.5	B
I4	60	91.0	50.0	B
I5	40	95.0	62.5	C
I6	25	97.5	75.0	C
I7	15	99.0	87.5	C
I8	10	100.0	100.0	C

Class A: I1, I2 (25% of items, 73% of value). Class B: I3, I4 (25% items, 18%). Class C: I5-I8 (50% items, only 9% of value).

Extra: single-period (newsboy) model - probabilistic demand

For perishable items bought once. Let C_u = profit lost per unit short (selling price - cost) and C_o = loss per unit left over (cost - salvage). Stock Q^* such that $\mathbf{P(demand \le Q^*) = C_u/(C_u + C_o)}$ (critical ratio).

Example: cost Rs 10, price Rs 25, salvage Rs 5. $C_u = 15$, $C_o = 5$, critical ratio = 15/20 = 0.75.

If demand is uniform between 100 and 200: $Q^* = 100 + 0.75 \times 100 = 175$ units.

Combining ABC with VED (control matrix)

	V (vital)	E (essential)	D (desirable)
A (high value)	tight control, small stock	moderate	very low stock
B	tight control	moderate	periodic review
C (low value)	keep large safety stock	bulk order	loose control

13. Applications, Limitations and Practice Problems

Applications of queuing theory

- Banks, supermarkets, petrol pumps, toll plazas: deciding the number of counters (servers)
- Hospitals and call centres: staffing; machine repair: number of repairmen; airports: runway and gate allocation; computer networks: buffer sizes

Limitations of the simple models

- Assume Poisson arrivals and exponential service with constant rates; real systems have peaks and non-exponential times
- Steady state may never be reached for short periods; balking, reneging and priorities complicate the maths - then **simulation** is used
- Other models (names only): (M/M/1):(N/FIFO) finite source, (M/G/1) Pollaczek-Khinchine, ($M/E_k/1$) Erlang service, priority queues, queues in series

Practice problems with answers

No.	Problem	Answer
1	M/M/1, λ = 4/hr, μ = 6/hr. Find P_0, L_s, L_q, W_s, W_q	1/3; 2; 1.333; 0.5 hr; 0.333 hr
2	M/M/1, λ = 3/hr, μ = 5/hr. L_s and P(4 or more in system)	1.5; $0.6^4 = 0.1296$
3	M/M/2, λ = 6/hr, μ = 4/hr each. Find P_0, L_q, W_q	1/7; 1.929; 0.321 hr
4	EOQ: D = 5000, C_0 = 200, C_h = 10. Q*, TC	447.2 units; Rs 4472
5	Same with shortage cost C_s = 40. Q*, S*, TC	500; 400 (backorder 100); Rs 4000
6	Production model, K = 10000, same data as 4. Q*, I_{max}, TC	632.5; 316.2; Rs 3162
7	Normal use 15/day, lead 8 days; max use 25/day, lead 10 days. ROL, buffer stock	250; 130 units

Check of problem 7: ROL = 25 × 10 = 250; average lead-time demand = 15 × 8 = 120; buffer = 250 - 120 = 130.

Formula flash cards

Queue ρ: M/M/1 $\rho = \lambda/\mu$; M/M/c $\rho = \lambda/(c\mu)$. *Always* check $\rho < 1$ before using steady-state formulas.

Time conversion: W in hours × 60 = minutes. $P(n \geq k) = \rho^k$ for M/M/1 infinite. $P(\text{wait} > 0) = 1 - P_0 = \rho$ for M/M/1.

Inventory: ordering cost = DC_0/Q, holding = $C_hQ/2$, equal at Q*. Reorder level = lead-time demand + buffer. ABC: A ≈ 70-80% value.

14. Quick Revision & Important Questions

Queues: $M/M/1$: $L_s = \lambda/(\mu-\lambda)$, $W_s = 1/(\mu-\lambda)$, $L_q = \lambda^2/\mu(\mu-\lambda)$; finite: $\lambda' = \lambda(1-P_N)$; $M/M/c$: P_0 formula, L_q formula; Little: $L = \lambda W$.
Inventory: $Q^* = \sqrt{(2DC_0/C_h)}$; with shortages multiply by $\sqrt{((C_h+C_s)/C_s)}$; production: divide inside root by $(1 - D/K)$; discount: compare total costs.

Likely questions: (1) Explain elements of a queuing system; (2) Derive P_n, L_s, W_s for $M/M/1$; (3) Solve $M/M/1$ and $M/M/c$ numericals; (4) Derive EOQ with and without shortages; (5) Production lot-size model; (6) Quantity-discount problem; (7) ABC analysis; (8) Buffer stock.

UNIT V

Replacement, Maintenance Analysis & DP

Handwritten study notes | B.Tech | Operations Research

Syllabus topics covered in this unit:

- Maintenance types; make-or-buy decision
- Types of replacement problems; economic life of an asset
- Simple probabilistic model: individual and group replacement
- Dynamic programming: Bellman's principle of optimality
- Shortest path, capital budgeting, solving an LP by DP

1. Maintenance - Introduction and Types

Maintenance is the set of activities needed to keep plant, machinery and equipment in good working condition so that production is not interrupted. The OR question is **how much maintenance, and when**, so that the sum of maintenance cost and breakdown loss is minimum.

Types of maintenance

Type	Meaning	Example
Breakdown (corrective)	repair only after failure occurs	bulb, fan, non-critical pump
Preventive	scheduled inspection, lubrication, part change before failure	oil change every 5000 km
Predictive (condition based)	monitor condition (vibration, temperature) and act when needed	bearing vibration analysis
Routine	daily cleaning, lubrication and checks	cleaning of machines
Planned / Scheduled	overhaul at fixed intervals based on records	annual shutdown maintenance

- Preventive maintenance (PM) reduces breakdown cost but increases maintenance cost. Optimum PM level = where **total cost (PM cost + breakdown cost)** is minimum
- PM is worthwhile when the failure rate **increases** with age and breakdown cost is much higher than PM cost

Preventive maintenance - cost trade-off example

A machine shop: each breakdown costs Rs 1500; each preventive maintenance (PM) visit costs Rs 300. Expected breakdowns per month depend on the number of PM visits per month:

PM visits k	Breakdowns	Breakdown cost	PM cost	Total cost
0	10	15000	0	15000
1	6	9000	300	9300
2	4	6000	600	6600
3	3	4500	900	5400
4	2.5	3750	1200	4950
5	2.4	3600	1500	5100

Minimum total cost Rs 4950 at 4 PM visits per month. More PM ($k = 5$) costs more than the breakdowns it saves.

Make or Buy decision

Should a firm **make** a component in-house or **buy** it from a supplier? Compare total cost of both.

Buy: $TC_b = p \times Q$ Make: $TC_m = F + v \times Q$ (F = fixed cost, v = variable cost per unit, p = purchase price)

Break-even quantity $Q_0 = F / (p - v)$: make if $Q > Q_0$, buy if $Q < Q_0$.

Example: buy price Rs 30; in-house fixed cost Rs 1,20,000, variable cost Rs 18/unit. Q_0 = 120000/(30 - 18) = 10,000 units. For requirement 15,000 units: buy = 4,50,000, make = 120000 + 270000 = 3,90,000 -> make. For 6,000 units: buy = 1,80,000, make = 2,28,000 -> buy.

Other factors: quality, capacity, delivery reliability, secrecy, supplier risk, labour and idle capacity.

2. Replacement Problems

Equipment gets worse with use: running (maintenance, repair, operating) cost goes up and resale value goes down. Replacement analysis finds the **best time to replace** so that the cost per unit time is minimum.

Types of replacement problems

- (i) Gradual deterioration: machines, vehicles - running cost increases with age (economic life problem)
- (ii) Sudden failure: electric bulbs, tubes, fuses, transistors - work perfectly till they fail suddenly: **individual vs group replacement** (probabilistic)
- (iii) Staff replacement: people leave/retire; recruitment and training replace them (probability of leaving by year)
- (iv) Obsolescence: equipment replaced by a better design even though working (new technology)

Common terms

- Capital (purchase) cost C; scrap/resale value S; running cost R(t) in year t; cumulative cost; **average annual cost** A(n)
- Economic life: the age n at which the average annual total cost is minimum

3. Economic Life of an Asset (money value ignored)

Average annual cost for n years: $A(n) = [\, C - S_n + \sum_{t=1}^{n} R(t) \,] / n$

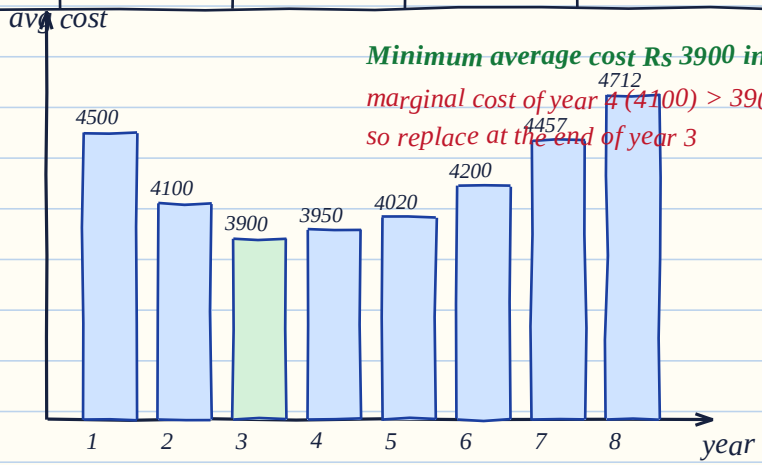
Rule 1: replace at the end of the year n for which A(n) is minimum.

Rule 2 (equivalent): keep the machine as long as the **marginal (next year) cost** $M(n+1) = R(n+1) + S_n - S_{n+1}$ is less than the present average cost A(n); replace when it exceeds A(n).

Example 1

A machine costs Rs 12,000. Running costs and resale values (end of year) are given. When should it be replaced?

Year n	Running cost	Cum. running	Resale S_n	Total cost $P - S_n$ + cum	Average cost	Marginal cost
1	1500	1500	9000	4500	4500	4500
2	1700	3200	7000	8200	4100	3700
3	2000	5200	5500	11700	3900	3500
4	2600	7800	4000	15800	3950	4100
5	3300	11100	3000	20100	4020	4300
6	4100	15200	2000	25200	4200	5100
7	5000	20200	1000	31200	4457	6000
8	6000	26200	500	37700	4712	6500



Minimum average cost Rs 3900 in year 3
marginal cost of year 4 (4100) > 3900
so replace at the end of year 3

Economic life = 3 years. Minimum average cost = Rs 3,900 per year (year 3).
Check by Rule 2: marginal cost in year 4 = 2600 + (5500 - 4000) = Rs 4100 > A(3) = 3900, while year 3's marginal cost 3500 < 3900.

Special case (no resale value, S = 0): replace when the running cost of the next year exceeds the

average total cost up to now. If $R(t) = a + bt$, optimal age $n^* = \sqrt{(2C/b)}$.

Example 1(a): no resale value (scrap = 0)

Machine cost Rs 5000; running costs year 1-7: 500, 700, 900, 1200, 1600, 2100, 2700.

Year	Running	Cumulative	Total (5000 + cum)	Average
1	500	500	5500	5500.0
2	700	1200	6200	3100.0
3	900	2100	7100	2366.7
4	1200	3300	8300	2075.0
5	1600	4900	9900	1980.0
6	2100	7000	12000	2000.0
7	2700	9700	14700	2100.0

Minimum average = Rs 1980 in year 5. Rule check: year 6 running cost 2100 > 1980 (average) so replace at the end of year 5; year 5 running cost 1600 < 1980 so keep up to year 5.

Replacement with money value (discounting)

- Let interest rate be i; discount factor $v = 1/(1 + i)$. Present worth of all costs over the n-year horizon is compared

Annual equivalent cost of keeping for n years: $W(n) = [\ C + \sum_{t=1}^{n} R(t)\, v^{t-1} - S_n\, v^n\ ] / [\ \sum_{t=1}^{n} v^{t-1}\ ]$

Replace at the n which minimises W(n). Rule: replace when the next year's running cost exceeds the weighted average of past costs (interest on capital is counted).

Reason: money available now is worth more than money later, so late-year costs are discounted; the economic life with discounting is usually **longer** than without.

Example 1(b): discounting at 10%

Machine cost Rs 5000; running cost years 1-5: 800, 1000, 1300, 1700, 2200; resale value at end of years 1-5: 3500, 2500, 1800, 1200, 800. Interest 10%, $v = 1/1.1 = 0.9091$.

n	Present worth of cost	$\sum v^{t-1}$	Annual equivalent W(n)
1	2618.2	1.000	2618.2
2	4643.0	1.909	2432.0
3	6431.1	2.736	2350.9
4	8241.1	3.487	2363.5
5	10066.6	4.170	2414.1

Minimum W(n) = Rs 2350.9 at n = 3, so replace every 3 years. (Here running costs are discounted to year-start values and S_n is discounted n years.)

Replacement because of a new, better machine (obsolescence)

Compare **annual cost of the old (defender)** with the minimum average cost of the **new (challenger)**. Replace the old machine now if its next-year cost exceeds the challenger's minimum average cost.

4. Items that Fail Completely - Probabilistic Replacement

Bulbs, tubes, fuses and many electronic parts work till they fail without warning. A **mortality table** gives p_i = probability that an item fails in the i-th period ($\sum p_i = 1$).

- Mean life = $\sum i \cdot p_i$. Average number of failures per period (steady state) = N / mean life
- Individual replacement: replace each item when it fails (cost c_i each)
- Group replacement: replace ALL items at fixed intervals (cost c_g each, $c_g < c_i$) and also replace individual failures in between

Number of failures in each period

$N_0 = N$ (all new at time 0); $N_t = N\,p_t + N_1\,p_{t-1} + N_2\,p_{t-2} + \ldots + N_{t-1}\,p_1$ (renewal equation)
Group policy cost for interval t: $C(t) = [\,N\,c_g + c_i \sum_{k=1}^{t-1} N_k\,] / t$. Choose t with minimum C(t); compare with individual-policy cost per period = $c_i\,N$ / mean life.

Example 2

1000 bulbs. Failure probability per week: 0.10, 0.15, 0.20, 0.25, 0.30 for weeks 1-5. Replacing one bulb on failure costs Rs 5; replacing all together costs Rs 2 per bulb. Find the best policy.

Week i	1	2	3	4	5
p_i	0.10	0.15	0.20	0.25	0.30

Mean life = 1(0.1) + 2(0.15) + 3(0.2) + 4(0.25) + 5(0.3) = 3.5 weeks. **Individual policy:** failures per week = 1000/3.5 = 285.7; cost = 285.7 × 5 = Rs 1428.6 per week.

Failures in successive weeks: $N_1 = 1000(0.10) = 100$; $N_2 = 1000(0.15) + 100(0.10) = 160$; $N_3 = 1000(0.20) + 100(0.15) + 160(0.10) = 231$; $N_4 = 317.1$; $N_5 = 423.4$.

Group interval t (weeks)	Cum. failures before t	Total cost Rs	Cost per week Rs
1	0	2000	2000.0
2	100	2500	1250.0
3	260	3300	1100.0
4	491	4455	1113.8
5	808	6040	1208.1

Best group policy: replace ALL bulbs every 3 weeks (cost Rs 1100 per week, the minimum).

Individual policy costs Rs 1428.6 per week, so group replacement saves 1428.6 - 1100 = Rs 328.6 per week. Interval 4 gives 1113.75 (higher), so t = 3.

Rule: the optimum group interval is the **last t for which the cost per period decreases**, i.e. replace when the cost of the individual policy for the next period ($c_i N_t$...) exceeds the average cost to date.

Example 3: another mortality table

N = 2000 items; p = 0.05, 0.15, 0.30, 0.50 (weeks 1-4); individual cost Rs 4, group cost Rs 1.5 per item. Mean life = 0.05 + 0.30 + 0.90 + 2.00 = 3.25 weeks; individual policy = 2000 × 4/3.25 = Rs 2461.5 per week.

$N_1 = 100$; $N_2 = 2000(0.15) + 100(0.05) = 305$; $N_3 = 2000(0.30) + 100(0.15) + 305(0.05) = 630.25$.

Interval t	Failures before t	Cost = 3000 + 4 × failures	Per week
1	0	3000	3000
2	100	3400	1700
3	405	4620	1540
4	1035.25	7141	1785.25

Best: group replacement every 3 weeks (Rs 1540/week) which is much lower than Rs 2461.5/week for individual replacement.

Policy	When to use	Condition
Individual	item fails rarely, group replacement cost is not much less	$c_g \approx c_i$
Group	many identical items, failure rate rises with age	$c_g << c_i$

Staff replacement (idea)

Treat the number of employees leaving in year i as the failure distribution. Using N_t as above, find how many people to recruit and train each year, and compare **group (batch) recruitment** with individual replacement.

Example 4: staff replacement

A firm has 1000 employees. The probability of an employee leaving in year i of service: 0.10, 0.20, 0.30, 0.40 (years 1-4; all leave by year 4). Mean service = 0.1 + 0.4 + 0.9 + 1.6 = 3 years, so in the long run about 1000/3 = 333 recruits per year are needed.

Year t	1	2	3	4
Leavers N_t	100.0	210.0	341.0	506.1

$N_1 = 100$, $N_2 = 1000(0.2) + 100(0.1) = 210$, $N_3 = 300 + 20 + 21 = 341$, $N_4 = 400 + 30 + 42 +$

34.1 = 506.1. The number of leavers oscillates and settles at about 333, so a steady hiring plan of ~333 per year (with batch training) is adopted.

Replacement because of obsolescence - quick example

Old machine: next-year operating cost Rs 4000. New machine: minimum average annual cost Rs 3500 (from its own table). Since 4000 > 3500 the old machine should be replaced now. If its cost were Rs 3200, keep it one more year.

5. Dynamic Programming (DP) - Introduction

DP is a method for solving **multistage (sequential) decision problems** by breaking the big problem into smaller sub-problems (stages), solving them one at a time and combining the results. It was developed by **Richard Bellman** (1950s).

Bellman's principle of optimality: An optimal policy has the property that, whatever the initial state and the initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision.

DP terminology

- Stage: a point at which a decision is taken (period, activity, plant)
- State (s_n): information describing the situation at a stage (resources left, node reached)
- Decision variable (d_n) and return $r_n(s_n, d_n)$: choice made and the immediate gain/cost
- Transition: $s_{n-1} = T(s_n, d_n)$; Optimal value function: $f_n(s)$ = best total return from stage n to the end

Recursive (functional) equation: $f_n(s_n) = \text{opt over } d_n \{ r_n(s_n, d_n) + f_{n-1}(s_{n-1}) \}$

Backward recursion starts from the last stage and works back to the first (most common); forward recursion starts at stage 1.

Why the principle works (idea)

Suppose the best route from A to D passes through node B. Then the part of that route from B to D must itself be the best route from B to D - otherwise replacing it by a better B-to-D route would improve the whole route, a contradiction. So we may solve small problems from the end and build up the answer.

DP type	Meaning
Deterministic	state transitions and returns are known with certainty
Probabilistic (stochastic)	next state is random with known probabilities; expected values are optimised
Discrete / continuous	stages or states take discrete values / continuous values
Forward / backward	recursion from first stage / from last stage

Characteristics of a DP problem

- Problem divides into stages with a decision at each stage; each stage has a number of states
- The decision changes the state (transformation); the effect of the past is summarised in the present state (**Markov / memoryless** property)
- Principle of optimality holds; recursion gives the final answer

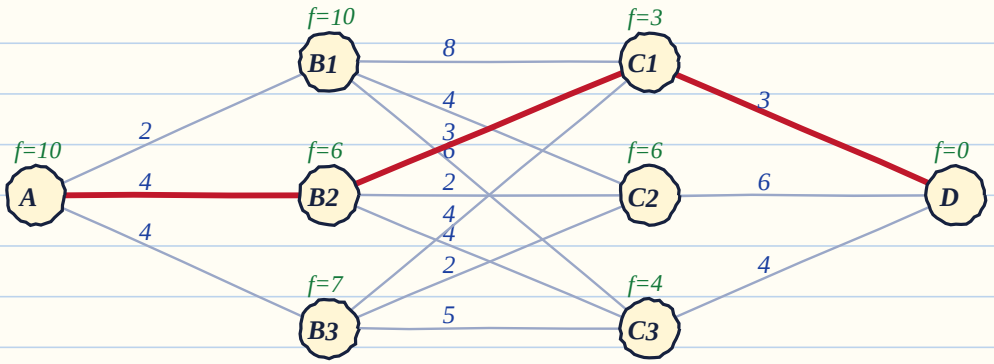
Applications

- Shortest/longest route, capital budgeting, resource allocation, inventory and production scheduling, equipment replacement, cargo loading (knapsack), LP problems

6. DP Application 1: Shortest Path Problem

Find the least-cost route from origin A (stage 0) to destination D through intermediate nodes.

Work **backwards**: f(node) = minimum of [arc cost + f(next node)].



Stage	Node	Choices (arc + f of next)	f(node)	Best next
3	C1 / C2 / C3	only D: 3, 6, 4	3 / 6 / 4	D
2	B1	8+3=11, 4+6=10, 6+4=10	10	C2 or C3
2	B2	3+3=6, 2+6=8, 4+4=8	6	C1
2	B3	4+3=7, 2+6=8, 5+4=9	7	C1
1	A	2+10=12, 4+6=10, 4+7=11	10	B2

Shortest path: A -> B2 -> C1 -> D with total length 4 + 3 + 3 = 10.

Forward check: f(A) = min(12, 10, 11) = 10. The red path in the figure is the optimal policy.

A second shortest-path example (table method)

Nodes 1-6 with arc lengths: 1-2: 4, 1-3: 2, 2-4: 5, 2-5: 3, 3-4: 1, 3-5: 6, 4-6: 2, 5-6: 2.

Backward:

Node	Choices	f(node)	Next
6	destination	0	-
5	2 + 0	2	6
4	2 + 0	2	6
3	1+2 = 3, 6+2 = 8	3	4
2	5+2 = 7, 3+2 = 5	5	5
1	4+5 = 9, 2+3 = 5	5	3

Shortest path 1 -> 3 -> 4 -> 6, length 2 + 1 + 2 = 5.

7. DP Application 2: Capital Budgeting Problem

A company has **Rs 5 crore** to allocate among three plants (stages). Returns $r_j(x)$ (Rs crore) from investing x crore (whole units) in plant j are given. Maximise total return.

Investment x	0	1	2	3	4	5
$r_1(x)$ Plant 1	0	3	7	9	12	13
$r_2(x)$ Plant 2	0	5	10	15	17	18
$r_3(x)$ Plant 3	0	5	6	8	9	10

Stage 1 and Stage 2

Stage 1: $f_1(x) = r_1(x)$ (all remaining money to Plant 1). Stage 2: $f_2(x)$ = max over k of [$r_2(k) + f_1(x - k)$].

State x	0	1	2	3	4	5
$f_1(x)$	0	3	7	9	12	13
$f_2(x)$	0	5	10	15	18	22
best k for plant 2	0	1	2	3	3	3

Example calc: $f_2(4)$ = max[0+12, 5+9, 10+7, 15+3, 17+0] = max[12, 14, 17, 18, 17] = 18 with k = 3. Stage 3: $f_3(5)$ = max over k of [$r_3(k) + f_2(5 - k)$]:

k (to plant 3)	0	1	2	3	4	5
$r_3(k) + f_2(5-k)$	0+22=22	5+18=23	6+15=21	8+10=18	9+5=14	10+0=10

Optimal allocation: Plant 3 gets 1 (return 5); remaining 4 -> Plant 2 gets 3 (return 15); remaining 1 -> Plant 1 gets 1 (return 3).

Maximum total return = 5 + 15 + 3 = Rs 23 crore. (backward tracing of the best k values)

8. DP Application 3: Cargo Loading (Knapsack) Problem

A vehicle of capacity 5 tonnes can carry three types of item (any number of each): weights 2, 3, 1 tonnes and values Rs 65, 80, 30. Maximise value. Stage = item type, state = capacity left.

Capacity x	0	1	2	3	4	5
$f_1(x)$: item 1 only	0	0	65	65	130	130
$f_2(x)$: items 1, 2	0	0	65	80	130	145
$f_3(x)$: items 1, 2, 3	0	30	65	95	130	160

Recursion: $f_j(x) = \max$ over $k \geq 0$ of $[\, k\, v_j + f_{j-1}(x - k\, w_j) \,]$. Tracing back from $f_3(5) = 160$: item 3 once (30, weight 1) and item 1 twice (130, weight 4). Maximum value = Rs 160.

Inventory/production example idea

Stage = month, state = stock at start of month, decision = production quantity, return = production + holding cost. Backward recursion $f_n(s) = \min [\, c(x) + h(s + x - d_n) + f_{n+1}(s + x - d_n) \,]$ gives the cheapest production plan.

9. DP Application 4: Solving an LP Problem by DP

Any LP with n variables can be treated as an n-stage problem: stage j chooses x_j; the **state** is the amount of each resource still unused.

Maximise $Z = 3x_1 + 5x_2$ s.t. $x_1 \leq 4$, $2x_2 \leq 12$, $3x_1 + 2x_2 \leq 18$, $x_1, x_2 \geq 0$.

- Stage 2 (x_2): state $s = 18 - 3x_1$ (capacity left in the 3rd constraint). $f_2(s) = \max 5x_2$ with $2x_2 \leq 12$ and $2x_2 \leq s \Rightarrow x_2 = \min(6, s/2)$, $f_2(s) = 5 \min(6, s/2)$
- Stage 1 (x_1): $f_1 = \max_{0 \leq x1 \leq 4} [\, 3x_1 + 5 \min(6, (18 - 3x_1)/2) \,]$
- For $x_1 \leq 2$: min = 6, value $= 3x_1 + 30$ (increasing). For $2 \leq x_1 \leq 4$: value $= 3x_1 + 5(9 - 1.5x_1) = 45 - 4.5x_1$ (decreasing)

Best at $x_1 = 2$: $x_2 = \min(6, 12/2) = 6$, $Z = 6 + 30 = 36$. Same as the simplex/graphical solution.

DP is useful for non-linear and integer problems too, but suffers from the **curse of dimensionality**: the number of states grows rapidly with the number of resources.

Point	Linear programming	Dynamic programming
Nature	single-shot optimisation	multi-stage, sequential
Method	simplex (standard algorithm)	no standard formula; recursion fitted to problem
Objective/constraints	must be linear	can be non-linear, integer, stochastic
Limit	very large m, n manageable	dimensionality problem

9. Summary Tables for Revision

Problem	Method / formula	Decision rule
Economic life (no interest)	$A(n) = [C - S_n + \sum R]/n$	min A(n); replace if $M(n+1) > A(n)$
Economic life (interest i)	$W(n)$ with $v = 1/(1+i)$	min $W(n)$
Sudden failure	$N_t = Np_t + \sum N_k p_{t-k}$	compare group C(t) vs individual cost
Make or buy	$Q_0 = F/(p - v)$	make if $Q > Q_0$
Shortest path (DP)	$f(i) = \min[d_{ij} + f(j)]$	backward from destination
Capital budgeting (DP)	$f_n(x) = \max[r_n(k) + f_{n-1}(x-k)]$	trace best k backwards

Typical exam steps

- Replacement: always tabulate year, running cost, cumulative cost, resale, total, average - the minimum average is the answer; verify with the marginal-cost rule
- Group replacement: compute failures per period using the renewal formula first; then group cost per period for t = 1, 2, ...; stop when cost starts to rise
- DP: define stages, states, decision, return and recursion **before** computing; show the table for each stage; trace back the optimal policy

Common mistakes

- Forgetting to subtract the resale value in A(n); using running cost instead of **cumulative** running cost
- In group replacement, counting the failures of the interval's last period as individual replacements (they are replaced as part of the group)
- In DP, working forward when the recursion was set up backward; not tracing back to get the actual decisions

10. Question Bank with Short Answers

2-mark questions

- Define economic life of an asset. -> The age at which the average annual cost of owning and running it is minimum.
- What is group replacement? -> Replacing all items together at fixed intervals, plus replacing individual failures in between.
- State Bellman's principle of optimality. -> Remaining decisions of an optimal policy must be optimal for the state reached after the first decision.
- What are the stages and states in DP? -> Stage = decision point; state = description of resources/position at that stage.
- Give two types of maintenance. -> Preventive (before failure) and breakdown/corrective (after failure).
- What is the curse of dimensionality? -> State space grows exponentially with the number of state variables, making DP tables huge.

5-mark questions

- Types of replacement problems with one example each; make-or-buy decision with break-even analysis.
- Explain the individual and group replacement policies and when group replacement is better.
- Formulate the shortest path problem as a DP and write the recursion.

10-mark questions

- Economic life from a table of running costs and resale values (with and without interest).
- Group replacement of items from a mortality table with cost comparison.
- Capital budgeting / cargo loading / shortest path by DP with full stage tables; solve a small LP by DP.

Study plan: first master the three table procedures (economic life table, renewal-equation table, stage table of DP). Practise each with two different data sets; marks are given for the layout of the table and the final decision statement.

11. Quick Revision & Important Questions

Replacement: $A(n) = [C - S_n + \sum R]/n$ minimum; replace when marginal cost > average cost. Group replacement: $C(t) = [N c_g + c_i \sum N_k]/t$; $N_t = N p_t + \sum N_k p_{t-k}$.
Make/buy: $Q_0 = F/(p - v)$. DP: $f_n(s) = \text{opt}\{ r + f_{n-1}(s') \}$; backward recursion; shortest path 10; capital budgeting 23.

Likely questions: (1) Types of maintenance; make-or-buy; (2) Types of replacement problems; (3) Find economic life of a machine from a cost table; (4) Individual vs group replacement for bulbs; (5) Bellman's principle with example; (6) Shortest path by DP; (7) Capital budgeting by DP; (8) Solve a small LP by DP.